



Convert a Word or ODT document into a static HTML site compatible with MkDocs using Gemini 3 and ChatGPT 5.2

Serge Tahé, January 2026

This document is a translation of the original [French version](#), generated using [an automated tool](#). Some expressions or technical terms may not fully reflect the original meaning.

This site was created using the [Word or ODT -> HTML] converter developed by Gemini 3 and ChatGPT 5.2 in January 2026 and described in this document.

Using the Gemini/ChatGPT converter, several websites have been generated from LibreOffice ODT documents and Word documents:

Java

- [\[Introduction to the Java Language \(1998\)\]](#);
- [\[Introduction to Web Programming in Java with Servlets and JSP Pages \(2002\)\]](#);
- [\[Introduction to Struts 1.x through Examples \(2003\)\]](#);
- [\[The Basics of MVC Web Development in Java Through Examples \(2006\)\]](#);
- [\[Java 5 Persistence Through Practice \(2007\)\]](#);
- [\[Building a Java EE Web Service with the NetBeans 6.5 IDE and the GlassFish Java EE Server \(2009\)\]](#);
- [\[Introduction to Struts 2 Through Examples \(2012\)\]](#);
- [\[Introduction to Java Server Faces, PrimeFaces, and PrimeFaces Mobile with Examples \(2012\)\]](#);

- [[Introduction to Java EE with the NetBeans IDE and the GlassFish Application Server \(2012\)](#)];
- [[A Client/Server Example - AngularJS 1.x / Spring 4 \(2014\)](#)];
- [[Introduction to the Java Language and the Spring Ecosystem Through a Case Study \(2015\)](#)];
- [[Introduction to Spring MVC and Thymeleaf through examples \(2015\)](#)]
- [[Working with a relational database using the Spring ecosystem \(2015\)](#)];
- [[Introduction to RxJava. Application to Swing and Android environments \(2016\)](#)];

Android

- [[Introduction to Android Tablet Programming with Android Studio \(2016\)](#)];

.NET

- [[Introduction to VB.NET through Examples \(2004\)](#)];
- [[Web Development with ASP.NET 1.1 \(2004\)](#)];
- [[Spring IoC for .NET \(2005\)](#)];
- [[Creating a Three-Tier Web Application with Spring.NET and VB.NET \(2005\)](#)];
- [[Learning C# Version 3.0 with the .NET Framework 3.5 \(2008\)](#)];
- [[Building a Three-Tier Web Application with ASP.NET 2.0, C#, Spring.NET, and NHibernate \(2010\)](#)];
- [[Introduction to the NHibernate Framework for the .NET Platform \(2011\)](#)];
- [[Introduction to Entity Framework 5 Code First Through Examples \(2012\)](#)];
- [[Introduction to the ASP.NET MVC Framework through Examples \(2013\)](#)];

JavaScript

- [[A Client/Server Example - AngularJS 1.x / Spring 4 \(2014\)](#)];
- [[Introduction to the ECMAScript 6 language through examples \(2019\)](#)];
- [[Introduction to the VUE.JS framework through examples \(2019\)](#)];
- [[Introduction to the NUXT.JS framework through examples \(2019\)](#)];

PHP

- [[MVC Development Methodology for a PHP4 Web Application \(2004\)](#)];
- [[Introduction to the PHP5 language through examples \(2011\)](#)];
- [[Introduction to the PHP7 language through examples \(2019\)](#)];

Python

- [[Introduction to Python 2.7 through Examples \(2012\)](#)];
- [[Introduction to the Python Language and the Flask Web Framework Through Examples \(2020\)](#)];
- [[Generate a Python script using AI tools \(2025\)](#)];

VBScript

- [[Introduction to VBScript \(2002\)](#)];

SQL

- [[Introduction to the SQL language with the Firebird DBMS \(2006\)](#)];

These are all old courses consisting of several hundred pages. The size of the ODT or DOCX document does not matter to the Gemini/ChatGPT converter. What matters to it are the structures found within them (see chapter: [Examples in this document](#)).

1. Introduction

The PDF of this document is available | [HERE](#) |.

The Gemini 3 / ChatGPT 5.2 converters [Word or ODT → HTML] are available | [HERE](#) |.

This document provides you with two converters:

- a LibreOffice ODT to HTML converter;
 - a Word DOCX to HTML converter;
-

In this article, we aim to provide readers with a Python converter for converting Word or ODT documents into a static HTML site. This converter was initially built by the [Gemini 3](#) AI and then by ChatGPT 5.2. It took these two AIs 356 iterations to produce the converter for this document. This involved several weeks of debugging. Gemini 3 did all the work at the beginning. It took several dozen iterations just to get a first version that was roughly correct. Then, I regularly added a new problem posed by new ODT documents. And at that point, Gemini often regressed. That is, what worked at step N no longer worked at subsequent steps. So I proceeded as follows: as soon as Gemini produced a converter that solved one of my problems, I made it a reference version and told Gemini about it. I saved this reference locally. Then, when I saw Gemini regressing for too long, I would ask it to revert to the last known reference version by providing it with the code. That's how, little by little, we built this converter together: me telling it what I wanted—essentially by pointing out the anomalies I found in the generated HTML site—and it producing the requested code.

I used [Gemini 3](#) with a pro license at 22 euros/month and ChatGPT 5.2 as well.

Gemini / ChatGPT will generate two Python scripts:

- [convert] to convert the LibreOffice ODT document or the Word DOCX document into an MkDocs site;
- [build] to convert the MkDocs site into a static HTML site;

I never looked at the generated code. I wanted to treat them as black boxes. You don't need to be a Python developer to follow this tutorial—or even a developer at all.

When I suggested improvements to the converter to the Gemini 3 AI, it sometimes got stuck. It couldn't make the requested improvements. At that point, we were on stable version V316. To move forward, I then gave this working version to ChatGPT 5.2 and asked it to make the desired improvements. ChatGPT correctly modified Gemini's code to meet my new requests. That's why I consider these two AIs to be the ones that generated the converter.

After that, I continued to use this technique. When one AI got stuck on a feature, I gave the other AI the latest known stable version.

The two AIs have different methods for delivering the requested Python code:

- Gemini provides the generated script code on the query page. You then need to copy and paste this code;
- ChatGPT provides a download link for the generated script;

Beyond 1,000 lines of generated code, Gemini showed significant weaknesses. Due to technical limitations specific to this AI, it was unable to display the entire generated code on the query page. Very often, lines of code were missing. Because of this limitation, at a certain point it was no longer possible to use Gemini 3. And so it was ChatGPT 5.2 that finished writing the script.

ChatGPT also generated the Word-to-HTML converter. I gave it the working ODT-to-HTML converter and asked it to adapt it for a Word document. It did so in 18 iterations. This is a common trait of both of these AIs: they understand the Python scripts we give them very well and can make improvements to them. For me, this is the best way to work with them.

2. Examples from this document

I'd like to write a short article. The interactions between an AI and a user are presented in the article [[Generating a Python Script with AI Tools](#)], which I'll refer to from now on as [\[ref1\]](#). Interactions with Gemini and ChatGPT will only be mentioned briefly. In any case, it was impossible to present all the iterations.

I will now present examples of the specific features of my ODT/DOCX documents that the Gemini/ChatGPT converter handles correctly. It is this very document that we will submit for HTML conversion to the Gemini/ChatGPT script. We'll see what it does with it.

2.1. Lists

The Gemini/ChatGPT converter can handle bulleted and numbered lists, even if they are nested:

2.1.1. Bulleted lists

- Item 1;
- Item 2;
- Item 3;
 - Item 3.1;
 - Item 3.1.1
 - Element 3.1.2
 - Element 3.1.2.1
 - Element 3.1.2.2
 - Element 3.2;
- Element 4;

2.1.2. Numbered lists

1. Element 1;
2. Element 2;
 - a. Element 2.1
 - i. Element 2.1.1
 1. Element 2.1.1.1
 2. Element 2.1.1.2
 - ii. Element 2.1.2
 - b. Element 2.2
3. Element 3;

2.1.3. Mixed lists 1

- Element 1;
- Element 2;
- Element 3;
 - Element 3.1;
 1. Element 3.1.1
 2. Element 3.1.2
 - Element 3.1.2.1
 - Element 3.1.2.2
 - Element 3.2;
- Element 4;

2.1.4. Mixed lists 2

1. Element 1;
2. Element 2;
 - a. Element 2.1
 - i. Element 2.1.1
 - Element 2.1.1.1
 - Element 2.1.1.2
 - ii. Element 2.1.2
 - b. Element 2.2
3. Element 3;

2.1.5. Manually numbered lists

"Manually defined" means that the user sets the number of the first paragraph in a numbered list using the following sequence: [right-click on the numbered paragraph / set the numbering value / start a new list / set the value].

I start a list with a number other than 1.

6. Element 6
7. Element 7

Here I break the list to say something, but then I want to continue the numbering.

8. Item 8
9. Item 9

Then I start a new numbered list:

11. item 11
12. item 12

2.2. Code blocks

My courses contain a lot of code blocks. These are often formatted code (bold, keyword colors) from IDEs (Eclipse, PyCharm, WebStorm, NetBeans). This formatted code is rendered identically by the converter.

When the code is not formatted (code from Notepad or similar), the Gemini/ChatGPT converter recognizes it (Java, C#, XML, HTML, etc.) using language keywords specified in a configuration file. When it recognizes a language, it inserts a marker (fence) for MkDocs so that MkDocs can adapt the code's syntax highlighting to the language used in the code block.

2.2.1. Enriched code blocks (Eclipse, Visual Studio, etc.)

Here are code blocks enhanced by various IDEs:

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplicationContext;
8.
9. public class Demo01 {
10.
```

```

11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // Retrieve the Spring context
14.         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15.         // Retrieve the beans
16.         Person p01 = ctx.getBean("person_01", Person.class);
17.         Person p02 = ctx.getBean("person_02", Person.class);
18.         List<Person> club = ctx.getBean("club", new ArrayList<Person>().getClass());
19.         Apartment apt01 = ctx.getBean(Apartment.class);
20.     ...

```

C#

```

11. using System;
12.
13. namespace Chap1 {
14.     class Taxes {
15.         static void Main(string[] args) {
16.             // Data arrays needed to calculate the tax
17.             decimal[] limits = { 4962M, 8382M, 14753M, 23888M, 38868M, 47932M,
18.                 0M };
19.             decimal[] coeffR = { 0M, 0.068M, 0.191M, 0.283M, 0.374M, 0.426M,
20.                 0.481M };
21.             decimal[] coeffN = { 0M, 291.09M, 1322.92M, 2668.39M, 4846.98M,
22.                 6883.66M, 9505.54M };
23.
24.             // retrieve marital status
25.             bool OK = false;
26.             string response = null;
27.             while (!OK) {
28.                 Console.WriteLine("Are you married (Y/N)?");
29.                 response = Console.ReadLine().Trim().ToLower();
30.                 if (response != "y" && response != "n")
31.                     Console.Error.WriteLine("Incorrect answer. Please try
32. again");
33.                 else OK = true;
34.             } //while
35.             bool marie = response == "y";
36.     ...

```

Python

```

21. # -----
22. def display(string):
23.     # print string
24.     print("string=%s" % string)
25.
26.
27. # -----
28. def display_type(variable):
29.     # displays the variable type
30.     print("type[%s]=%s" % (variable, type(variable)))
31.
32.
33. # -----
34. def f1(param):
35.     # adds 10 to param
36.     return param + 10
37.
38.
39. # -----
40. def f2():
41.     # returns a tuple of 3 values
42.     return "one", 0, 100
43.
44.
45. # ----- main program -----
46. ...

```

PHP

```

31. <?php
32.
33. // strict types for function parameters
34. declare(strict_types=1);
35.
36. // global constants
37. define("HALF-RATE_INCOME_LIMIT", 1551);
38. define("INCOME_LIMIT_SINGLE_PERSON_FOR_REDUCTION", 21037);
39. define("INCOME_LIMIT_FOR_COUPLES_FOR_REDUCTION", 42074);
40. define("REDUCTION_VALUE_HALF_PORTION", 3797);
41. define("SINGLE_PERSON_DISCOUNT_LIMIT", 1196);
42. define("COUPLE_DEDUCTION_LIMIT", 1970);
43. define("COUPLE_INCOME_THRESHOLD_FOR_DEDUCTION", 2627);
44. define("SINGLE_TAX_CAP_FOR_DEDUCTION", 1595);
45. define("MAXIMUM_10_PERCENT_DEDUCTION", 12502);
46. define("MINIMUM_10_PERCENT_DEDUCTION", 437);
47.
48. // definition of local constants
49. $DATA = "taxpayersdata.txt";
50. $RESULTS = "results.txt";
51. $limits = array(9964, 27519, 73779, 156244, 0);
52. $coeffR = array(0, 0.14, 0.3, 0.41, 0.45);
53. $coeffN = array(0, 1394.96, 5798, 13913.69, 20163.45);
54.
55. // read data
56. $data = fopen($DATA, "r");
57. if (!$data) {
58.     print "Unable to open the data file [$DATA] for reading\n";
59.     exit;
60. }
61.
62. ...

```

ECMAScript

```

41. 'use strict';
42. // this is a comment
43. // constant
44. const name = "dupont";
45. // a screen display
46. console.log("name: ", name);
47. // an array with elements of different types
48. const array = ["one", "two", 3, 4];
49. // its number of elements
50. let n = array.length;
51. // a loop
52. for (let i = 0; i < n; i++) {
53.     console.log("array[" + i + "] = ", array[i]);
54. }
55. // Initializing 2 variables with the contents of an array
56. let [string1, string2] = ["string1", "string2"];
57. // concatenating the two strings
58. const string3 = string1 + string2;
59. // display result
60. console.log([string1, string2, string3]);
61. ...

```

VBScript

```

51. ' Calculating a taxpayer's tax
52. ' the program must be called with three parameters: married children salary
53. ' married: character 0 if married, N if unmarried
54. ' children: number of children
55. ' salary: annual salary without cents
56.
57. ' mandatory variable declaration
58. Option Explicit
59. Dim error
60.
61. ' retrieve the arguments and check their validity
62. Dim wife, children, salary

```

```

63. error = GetArguments(wife, children, salary)
64. ' error?
65. If error(0) <> 0 Then wscript.echo error(1) : wscript.quit error(0)
66.
67. ' Retrieve the data needed to calculate the tax
68. Dim limits, coeffR, coeffN
69. error = getData(limits, coeffR, coeffN)
70. ' error?
71. If error(0) ≠ 0 Then wscript.echo error(1) : wscript.quit 5
72.
73. ' Display the result
74. wscript.echo "tax=" & calculateTax(spouse,children,salary,limits,coeffR,coeffN)
75.
76. ' exit without error
77. wscript.quit 0

```

XML

```

61. <?xml version="1.0" encoding="utf-8" ?>
62. <configuration>
63.
64.   <configSections>
65.     <sectionGroup name="spring">
66.       <section name="context" type="Spring.Context.Support.ContextHandler, Spring.Core" />
67.       <section name="objects" type="Spring.Context.Support.DefaultSectionHandler, Spring.Core"
68.     />
69.   </sectionGroup>
70. </configSections>
71.
72. <spring>
73.   <context>
74.     <resource uri="config://spring/objects" />
75.   </context>
76.   <objects xmlns="http://www.springframework.net">
77.     <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
78.       <constructor-arg index="0" value="MySQL.Data.MySqlClient"/>
79.       <constructor-arg index="1"
80.       value="Server=localhost;Database=bdimpots;Uid=admimpots;Pwd=mdpimpots;"/>
81.       <constructor-arg index="2" value="select limite, coeffr, coeffn from tranches"/>
82.     </object>
83.     <object name="metier" type="Metier.ImpotMetier, ImpotsV7-metier">
84.       <constructor-arg index="0" ref="dao"/>
85.     </object>
86.   </objects>
87. </spring>
88. </configuration>

```

2.2.2. Blocks of raw code (plain text)

Here are some examples of raw code:

Execution results

Note that the code does not start on line 1.

```

7. C:\Data\st-2020\dev\python\cours-2020\python3-flask-2020\venv\Scripts\python.exe C:/Data/st-
8. 2020/dev/python/cours-2020/python3-flask-2020/bases/bases_01.py
9. name=dupont
10. list[0] = one
11. list[1] = two
12. list[2] = 3
13. list[3] = 4
14. [string1, string2, string1string2]
15. string = string1
16. type[4] = <class 'int'>
17. type[string1] = <class 'str'>

```

```

17. type[['one', 'two', 3, 4]] = <class 'list'>
18. type[has changed] = <class 'str'>
19. res1 = 14s
20. (res1, res2, res3) = [one, 0, 100]
21. list[0] = one
22. list[1] = 0
23. list[2] = 100
24. list[0] = 8
25. list[1] = 5
26. sum=13
27.
28. Process finished with exit code 0

```

This case required dozens of iterations. The converter could never find the line number of the first line of code. Finally, I asked Gemini how to format the code block so that it would recognize it. Here is what it replied:

Dans LibreOffice :

1. Placez votre curseur sur la **première ligne** de votre bloc de code numéroté (celle qui doit être le n°5).
2. Faites un **Clic-Droit > Paragraphe > Paragraphe...**
3. Allez dans l'onglet **Plan & numérotation** (ou *Outline & Numbering*).
4. Cochez la case **Recommencer la numérotation** (*Restart numbering at this paragraph*).
5. Cochez/Réglez **Commencer à** (*Start with*) sur **5**.
6. Cliquez sur **OK** et sauvegardez votre ODT.

Pourquoi ça marche ? Cette méthode oblige LibreOffice à inscrire un attribut explicite sur le paragraphe ou l'item de liste (`text:start-value="5"`), ce que le script détecte immédiatement. La méthode via l'icône de liste ("Personnaliser") modifie parfois seulement le style visuel sans changer la valeur logique sous-jacente dans le XML, surtout sur les documents importés de Word.

I followed these steps and it worked. I was using the method via the list icon (Customize). So you may need to modify certain elements of the ODT/DOCX document to get the result you want.

The converter can handle unnumbered code blocks.

```

list[0]=one
list[1]=0
list[2]=100
list[0]=8
list[1]=5
sum=13

```

Here are the same examples as in section 2.2.1, but without additional information. In this case, the keywords in the configuration file will guide the converter to the correct language.

Java

```

1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplicationContext;
8.

```

```

9. public class Demo01 {
10.
11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // Retrieve the Spring context
14.         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15.         // Retrieve the beans
16.         Person p01 = ctx.getBean("person_01", Person.class);
17.         Person p02 = ctx.getBean("person_02", Person.class);
18.         List<Person> club = ctx.getBean("club", new ArrayList<Person>().getClass());
19.         Apartment apt01 = ctx.getBean(Apartment.class);
20.         ...

```

C#

```

11. using System;
12.
13. namespace Chap1 {
14.     class Taxes {
15.         static void Main(string[] args) {
16.             // Data arrays needed to calculate the tax
17.             decimal[] limits = { 4962M, 8382M, 14753M, 23888M, 38868M, 47932M,
18.                 0M };
19.             decimal[] coeffR = { 0M, 0.068M, 0.191M, 0.283M, 0.374M, 0.426M,
20.                 0.481M };
21.             decimal[] coeffN = { 0M, 291.09M, 1322.92M, 2668.39M, 4846.98M,
22.                 6883.66M, 9505.54M };
23.
24.             // retrieve marital status
25.             bool OK = false;
26.             string response = null;
27.             while (!OK) {
28.                 Console.Write("Are you married (Y/N)?");
29.                 response = Console.ReadLine().Trim().ToLower();
30.                 if (response != "y" && response != "n")
31.                     Console.Error.WriteLine("Incorrect answer. Please try
32.                         again");
33.                 else OK = true;
34.             } //while
35.             bool marie = response == "y";
36.             ...

```

Python

```

21. # -----
22. def display(string):
23.     # print string
24.     print("string=%s" % string)
25.
26.
27. # -----
28. def display_type(variable):
29.     # displays the variable type
30.     print("type[%s]=%s" % (variable, type(variable)))
31.
32.
33. # -----
34. def f1(param):
35.     # adds 10 to param
36.     return param + 10
37.
38.
39. # -----
40. def f2():
41.     # returns a tuple of 3 values
42.     return "one", 0, 100
43.
44.
45. # ----- main program -----
46. ...

```

PHP

```
31. <?php
32.
33. // strict types for function parameters
34. declare(strict_types=1);
35.
36. // global constants
37. define("HALF-RATE_INCOME_LIMIT", 1551);
38. define("INCOME_LIMIT_SINGLE_PERSON_FOR_REDUCTION", 21037);
39. define("INCOME_LIMIT_FOR_COUPLES_FOR_REDUCTION", 42074);
40. define("REDUCTION_VALUE_HALF_PORTION", 3797);
41. define("SINGLE_PERSON_DISCOUNT_LIMIT", 1196);
42. define("COUPLE_DEDUCTION_LIMIT", 1970);
43. define("COUPLE_INCOME_THRESHOLD_FOR_DEDUCTION", 2627);
44. define("SINGLE_TAX_CAP_FOR_DEDUCTION", 1595);
45. define("MAXIMUM_10_PERCENT_DEDUCTION", 12502);
46. define("MINIMUM_10_PERCENT_DEDUCTION", 437);
47.
48. // definition of local constants
49. $DATA = "taxpayersdata.txt";
50. $RESULTS = "results.txt";
51. $limits = array(9964, 27519, 73779, 156244, 0);
52. $coeffR = array(0, 0.14, 0.3, 0.41, 0.45);
53. $coeffN = array(0, 1394.96, 5798, 13913.69, 20163.45);
54.
55. // read data
56. $data = fopen($DATA, "r");
57. if (!$data) {
58.     print "Unable to open the data file [$DATA] for reading\n";
59.     exit;
60. }
61.
62. ...
```

ECMAScript

```
41. 'use strict';
42. // this is a comment
43. // constant
44. const name = "dupont";
45. // a screen display
46. console.log("name: ", name);
47. // an array with elements of different types
48. const array = ["one", "two", 3, 4];
49. // its number of elements
50. let n = array.length;
51. // a loop
52. for (let i = 0; i < n; i++) {
53.     console.log("array[" + i + "] = ", array[i]);
54. }
55. // Initializing 2 variables with the contents of an array
56. let [string1, string2] = ["string1", "string2"];
57. // concatenating the two strings
58. const string3 = string1 + string2;
59. // display result
60. console.log([string1, string2, string3]);
61. ...
```

VBScript

```
51. ' Calculating a taxpayer's tax
52. ' the program must be called with three parameters: married children salary
53. ' married: character 0 if married, N if unmarried
54. ' children: number of children
55. ' salary: annual salary without cents
56.
57. ' mandatory variable declaration
58. Option Explicit
59. Dim error
60.
```

```

61. ' retrieve the arguments and check their validity
62. Dim wife, children, salary
63. error = GetArguments(wife, children, salary)
64. ' error?
65. If error(0) <> 0 Then wscript.echo error(1) : wscript.quit error(0)
66.
67. ' Retrieve the data needed to calculate the tax
68. Dim limits, coeffR, coeffN
69. error = getData(limits, coeffR, coeffN)
70. ' error?
71. If error(0) ≠ 0 Then wscript.echo error(1) : wscript.quit 5
72.
73. ' Display the result
74. wscript.echo "tax=" & calculateTax(spouse,children,salary,limits,coeffR,coeffN)
75.
76. ' exit without error
77. wscript.quit 0

```

XML

```

61. <?xml version="1.0" encoding="utf-8" ?>
62. <configuration>
63.
64.   <configSections>
65.     <sectionGroup name="spring">
66.       <section name="context" type="Spring.Context.Support.ContextHandler, Spring.Core" />
67.       <section name="objects" type="Spring.Context.Support.DefaultSectionHandler, Spring.Core"
68.     />
69.   </sectionGroup>
70. </configSections>
71.
72. <spring>
73.   <context>
74.     <resource uri="config://spring/objects" />
75.   </context>
76.   <objects xmlns="http://www.springframework.net">
77.     <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
78.       <constructor-arg index="0" value="MySQL.Data.MySqlClient"/>
79.       <constructor-arg index="1"
80.       value="Server=localhost;Database=bdimpots;Uid=admimpots;Pwd=mdpimpots;"/>
81.       <constructor-arg index="2" value="select limit, coeffr, coeffn from brackets"/>
82.     </object>
83.     <object name="metier" type="Metier.ImpotMetier, ImpotsV7-metier">
84.       <constructor-arg index="0" ref="dao"/>
85.     </object>
86.   </objects>
87. </spring>
88. </configuration>

```

HTML

```

71. <!DOCTYPE HTML>
72. <HTML>
73.   <head>
74.     <title>Laragon</title>
75.
76.     <link href="https://fonts.googleapis.com/css?family=Karla:400 " rel="stylesheet"
77.     type="text/css">
78.
79.     <style>
80.       HTML, body {
81.         height: 100%;
82.       }
83.
84.       body {
85.         margin: 0;
86.         padding: 0;
87.         width: 100%;
88.         display: table;
89.         font-weight: 100;
90.         font-family: 'Karla';

```



```

91.
92.         .container {
93.             text-align: center;
94.             display: table-cell;
95.             vertical-align: middle;
96.         }
97.
98.         .content {
99.             text-align: center;
100.            display: inline-block;
101.        }
102.
103.        .title {
104.            font-size: 96px;
105.        }
106.
107.        .opt {
108.            margin-top: 30px;
109.        }
110.
111.        .opt a {
112.            text-decoration: none;
113.            font-size: 150%;
114.        }
115.
116.        a:hover {
117.            color: red;
118.        }
119.    </style>
120. </head>
121. <body>
122.     <div class="container">
123.         <div class="content">
124.             <div class="title" title="Laragon">Laragon
125.
126.             <div class="info"><br />
127.                 Apache/2.4.35 (Win64) OpenSSL/1.1.0i PHP/7.2.11<br />
128.                 PHP version: 7.2.11   <span><a title="phpinfo()"
129. href="/?q=info">info</span></span><br />
130.                 Document Root: C:/myprograms/laragon-lite/www<br />
131.

```

2.3. Links

The Gemini / ChatGPT converter preserves external links in ODT / DOCX documents. For example, [Gemini 3](#) or [\[Generate a Python script with AI tools\]](#).

It can handle a link to a chapter [Link to a chapter](#)

A chapter reference: [2.1.2](#)

A reference to a preceding text marker (bookmark):

A reference to a text marker (bookmark) that follows:

2.4. Text formatting

The converter supports bold, italic, underlined, and highlighted text. It preserves the highlight color.

Text with words in **bold**, *italics*, underlined, or highlighted.

This also applies to links: [\[Generate a Python script with AI tools\]](#).

The **converter** also **handles character colors**.

It also handles the top and bottom borders of paragraphs.

- It also handles the top and bottom borders of paragraphs.

- It also handles the top and bottom borders of paragraphs.
- It also handles the top and bottom borders of paragraphs.

1. It also handles the top and bottom borders of paragraphs.
2. It also manages the top and bottom borders of paragraphs.
3. It also controls the top and bottom borders of paragraphs.

10. Text with words in **bold**, *italics*, underlined, or highlighted.
11. This also applies to links: [[Generate a Python script using AI tools](#)].
12. The **converter** also **handles character colors**.

- Text containing words in **bold**, *italics*, underlined, or highlighted.
- This also applies to links: [[Generate a Python script using AI tools](#)].
- The **converter** also **handles character colors**.

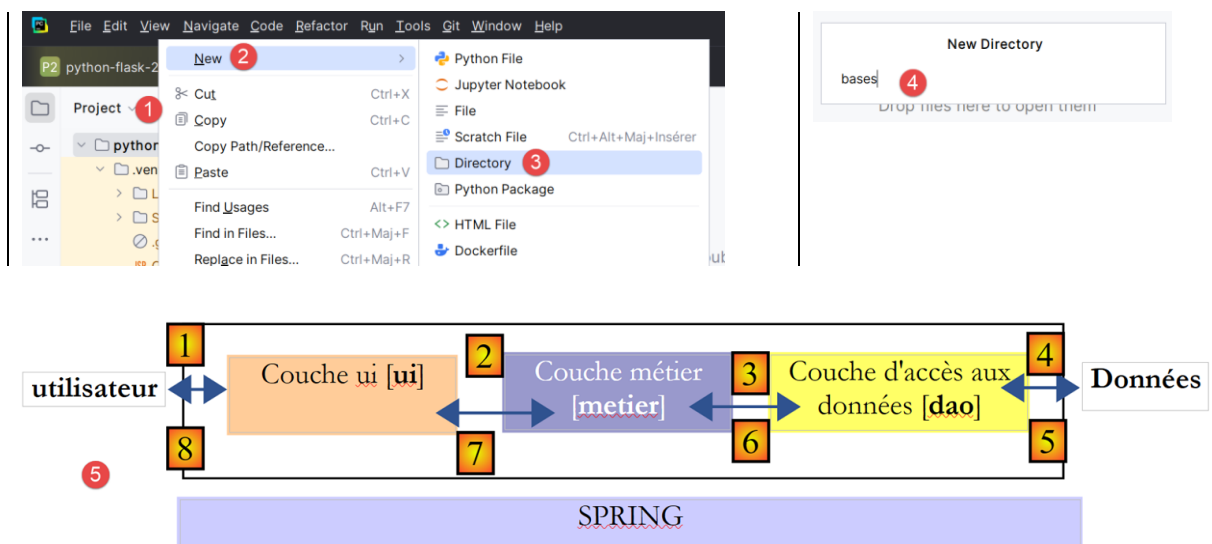
It also handles the paragraph background

- It also handles the paragraph background
1. It also handles the paragraph background

2.5. A title can also be enhanced.

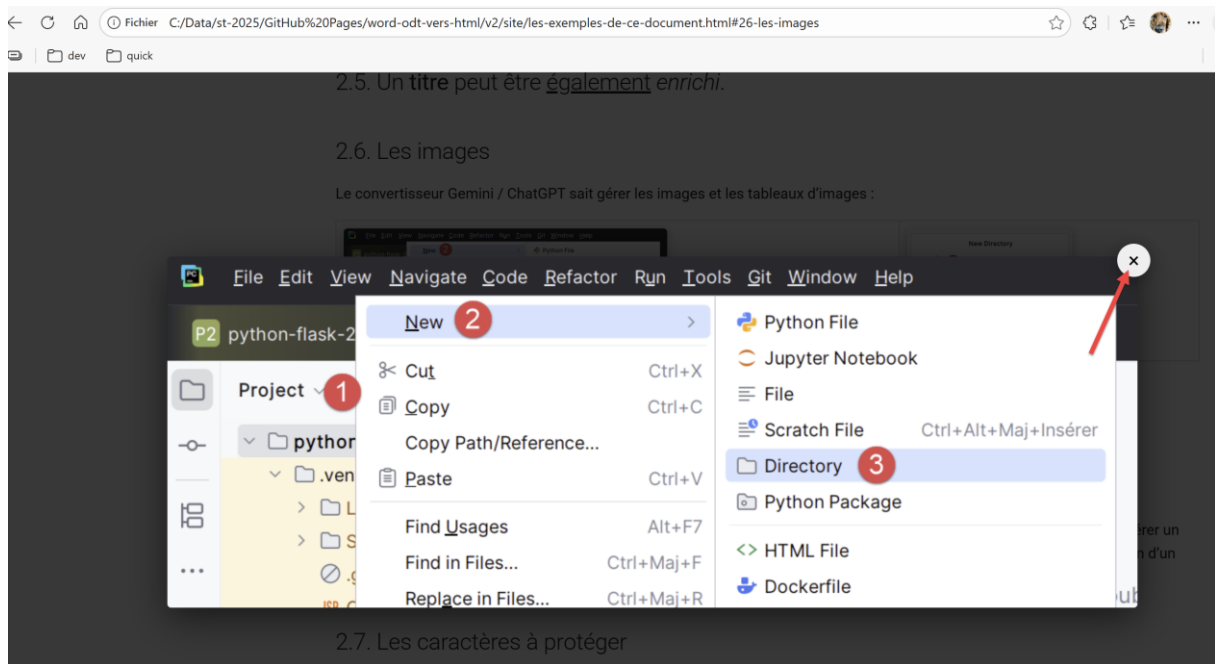
2.6. Images

The Gemini/ChatGPT converter can handle images and image tables:



In ODT documents, it is common to have drawings. Despite dozens of attempts, Gemini was unable to generate a script that would produce an image (such as a screenshot) of the drawing. Thus, in the example above, Image 5 is a screenshot of a drawing from an ODT document.

All images are clickable to enlarge. If you click on image [1-3] above, you get the following enlarged view:



2.7. Characters to protect

A MkDocs site¹ has pages whose content is not HTML but Markdown. If the ODT/DOCX document contains characters that exist in Markdown, they may be interpreted by MkDocs and therefore not render as expected. Here are two examples:

The asterisk `*` has a Markdown meaning. The following line may therefore be misinterpreted:

The tax I is then equal to **0.15*R – 2072.5*nbParts**.

Another example is when you want to insert a Markdown code block into your document like this:

```
1. # Word/ODT to HTML Site Converter (MkDocs)
2.
3.  🔗 **[See the generated demo site](https://stahe.github.io/word-odt-vers-html-janv-2026/)**
4.
5. ---
6.
7. ## 📄 Description
8.
9. This project aims to provide readers with a Python converter for converting Word or ODT
10. documents into a static HTML site.
11. When the ODT/DOCX document is suitable, the converter generates an HTML site using **MkDocs**
12. that has the professional look of sites produced by Pandoc.
13.
14. ## 🏠 Creation Context
15.
16. This converter was built entirely by the **Gemini 3** AI (with a pro subscription). It is the
17. result of successive iterations to finely manage the structure of ODT (OpenDocument Text)
18. documents.
19.
20. ## ✨ Features
21.
22. The `convert.py` script performs the following actions:
23.
24. * **ODT to Markdown Conversion**: Analyzes the `.odt` (XML) file to extract its structure.
25. * **Heading Management**: Automatically generates the Table of Contents (TOC) and sidebar
26. navigation.
```

¹ The MkDocs footnote

```
23. * **Code Blocks**: Automatic language detection, syntax highlighting, and **precise line  
    numbering** (`start-value` attributes).  
24. * **Lists**: Support for bulleted and numbered lists with correct indentation.  
25. * **Formatting**: Support for *bold*, *italics*, *underline*, and *highlighting* (while  
    preserving the original colors).  
26. * **Images**: Automatic extraction and embedding of images contained in the document.  
27. * **Configuration**: Customization via a `config.py` file (footer, Google Analytics, etc.).  
28.  
29. ## 🚀 Installation  
30.  
31. ### Prerequisites  
32.  
33. * Python 3.x  
34. * The following libraries:  
35.  
36. ```bash  
37. pip install odfpy unicode mkdocs mkdocs-material  
38.  
39. ```
```

2.8. Tables

A table can contain various types of content:

1	2		<pre>1. package istia.st.spring.core; 2. 3. import java.util.ArrayList; 4. import java.util.List; 5. 6. import org.springframework.context.ApplicationContext; 7. import org.springframework.context.support.ClassPathXmlApp licationContext; 8. 9. public class Demo01 { 10. 11.</pre>
3	4		<pre>1. # ----- 2. def display(string): 3. # displays string 4. print("string=%s" % string) 5. 6. 7. # ----- 8. def display_type(variable): 9. # displays the variable type 10. print("type[%s]=%s" % (variable, 11. type(variable))) 12.</pre>

The table can also contain links:

The Gemini / ChatGPT converter preserves external links in the ODT / DOCX document. For example, Gemini 3 or [Generate a Python script with AI tools]. It can handle a link to a chapter Link to a chapter	A reference to a chapter: 2.1.2 A reference to a preceding text marker (bookmark): A reference to a text marker (bookmark) that follows:
--	---

A table whose cells contain another table:

html:form	is used both to generate the HTML <form> tag and to provide information to the controller that will process this form:	
	action	URL where the form values will be sent
	name	name of the HTML form. This is also the name of the bean that will store the form's values
	type	Name of the class that must be instantiated to obtain the form storage bean
	Note that the method used to send form parameters (GET/POST) to the controller is not specified. This could be done using the method attribute. If this attribute is missing, the POST method is used by default.	
html:text	Used to generate the <input type="text" value="..."> tag:	
	property	name of the field in the form bean that will be associated with the input field. When the form is submitted to the server (client -> server), the bean field will take the value of the input field. When the form is displayed (server -> client), the value contained in the bean field is displayed in the input field.

It also manages the background of the paragraph within a table	
It also manages the background of the paragraph in a table	
2. It also handles the background of the paragraph within a table	

2.9. Footnotes

The Gemini² / ChatGPT converter handles footnotes. Here is another footnote³.

3. What's available online

Before getting started, we can ask the AI what free tools are available on the market for converting a Word or ODT document into a static HTML site.

Let's ask Gemini:

Existe-t-il sur internet un logiciel gratuit capable de convertir un document Word en site statique HTML ?	
--	--

Here is its response:

Yes, there are several free solutions, but the choice depends on your definition of a "static site" (a simple HTML page vs. a full website with navigation) and your technical skills. Here are the best options, categorized by method: 1. The "All-in-One" Solution (To Create a Fully Navigable Website) If you want to turn a long Word document (with chapters and subchapters) into a full website with a navigation menu on the left, this is the best option. Software: HelpNDoc (Free version for personal use) How it works: This is documentation creation software. You import your Word file, it analyzes the structure (Heading 1, Heading 2, etc.), and automatically generates a static, responsive HTML5 website (optimized for mobile devices).

² Gemini footnote

³ Another footnote

- **Pros:** Creates a real website with a menu, search bar, and index without writing a single line of code.
- **The downside:** The free version adds a small "Generated by HelpNDoc" banner at the bottom of the page.
- **2. The "Online Cleaner" Solution (For converting page by page)**

If you have a short document and just want to get the HTML code to put it on an existing page, online converters are ideal for cleaning up the "messy" code generated by Word.

- **Tool: WordToHTML.net** (or Word2CleanHTML)
- **How it works:**
 1. Copy your text from Word.
 2. Paste it into the visual editor on the left side of the site.
 3. The site generates clean HTML code (stripped of unnecessary Microsoft tags) in the right-hand window.

- **Pros:** Very simple, visual, removes the "fat" from Microsoft code.
- **The downside:** Ideal for a single page, not for generating the architecture of an entire site.

- **3. The "Pro/Developer" Solution (Best Quality)**

This is the method recommended by experts for obtaining ultra-clean, long-lasting code.

- **Software: Pandoc** (Command line)
- **How it works:** It's a small program without a graphical interface (to be used via the terminal/command prompt). It converts formats very accurately.
- **Sample command:** `pandoc my_document.docx -t html -o my_site.html`
- **Pros:** Perfect HTML5 code, free, open-source, handles images and mathematical formulas (LaTeX) very well.
- **Cons:** Requires familiarity with a command prompt (black screen).
- **4. The "DIY" Solution (Microsoft Word)**

You can do this directly from Word, but be careful with the result.

- **Method:** File > Save As > Select "**Filtered Web Page (*.htm; *.html)**".
- **Important:** Do not select "Single-file Web Page (.mht)"; be sure to choose "Filtered." The "Filtered" option reduces (slightly) the unnecessary code that Word usually adds.
- **The downside:** The code is often still bulky and difficult to modify later.

- **Quick Comparison Table**

Solution	Result Type	Code Quality	Ease	Ideal for...
HelpNDoc	Full website (with menu)	Good	Average	Manuals, theses, documentation
WordToHTML	Single page	Excellent	Very Easy	Blog posts, simple pages
Pandoc	Single page (raw)	Perfect	Difficult	Developers, clean archiving

Word (Native)	Single page	Poor	Easy	Quick troubleshooting
• My advice If your goal is to publish this document online in a professional manner, I suggest HelpNDoc if you're a beginner and want an immediate site structure. If you just want the plain text to paste it somewhere else (WordPress, etc.), use WordToHTML.net. Would you like me to walk you through the step-by-step process for one of these tools (for example, WordToHTML)?				

I've used [Pandoc] a few times, but I found it difficult to use. I recall that you had to "prepare" the Word document so that Pandoc could handle it properly. However, the result was excellent. **HelpNDoc** seems like an attractive option since it generates a website with a menu. I tested it on one of my Word documents, and the result wasn't satisfactory.

Now let's see if there is any conversion software for ODT documents:

Existe-t-il sur internet un logiciel gratuit capable de transformer un document ODT en site HTML statique ?

Gemini's response is as follows:

Yes, absolutely. As with Word, the ODT format (used by LibreOffice and OpenOffice) has its own solutions. Here are the best free options specific to ODT files, sorted by method: • 1. The "Extension" Solution (Best Built-in Quality) If you use LibreOffice or OpenOffice, the basic HTML export is (as with Word) rather poor. However, there is a fantastic free extension designed specifically for this purpose. • Tool: Writer2xhtml (Extension for LibreOffice) • How it works: 1. You download and install the extension in LibreOffice. 2. Go to File > Export, and a new option "XHTML 1.1 + MathML" (or similar) will appear. • Pros: The code is much cleaner than the standard export, and it handles mathematical formulas and complex formatting very well. • Cons: You need to install an extension (a one-time technical step). • 2. The "Pro / Command Line" Solution (Universal) This is the same tool as for Word, as it also excels with the ODT format. • Software: Pandoc • How it works: Via the command line. • Command: <code>pandoc my_file.odt -t html -o my_site.html</code> • Pros: It is undoubtedly the most powerful converter on the market for ODT to HTML. It "understands" the structure of the ODT document better than any other. • 3. The "Full Website with Navigation" Solution (Via HelpNDoc) HelpNDoc (mentioned earlier) is still the best free option for creating a real website with a left-hand menu. However, HelpNDoc doesn't always natively import the .odt format as well as the .docx format.

- **The trick:**
 1. Open your file in LibreOffice.
 2. Select "Save As..." and choose "**Word 2007-365 (.docx)**".
 3. Import this .docx file into HelpNDoc to generate your website.
- **Why?** This ensures that your chapter hierarchy (Heading 1, Heading 2) is correctly recognized to create the site menu.
- **4. The online solution (Quick cleanup)**

Some online tools directly accept ODT files and output clean HTML.

- **Tool: WordToHTML.net** (Also accepts ODT)
- **How it works:** You can upload your ODT file directly or paste your text.
- **Pros:** Very convenient for converting a single article without installing anything.
- **Summary of my advice for ODT:**
 1. **For a real website (documentation/thesis):** Convert your ODT to **DOCX** using LibreOffice, then use **HelpNDoc** (this is the most reliable method for getting a navigation menu).
 2. **For a simple, clean page:** Use the **Writer2xhtml** extension if you do this often, or **Pandoc** if you want to automate the task.

Would you like me to explain how to install the **Writer2xhtml** extension in LibreOffice?

I tested the LibreOffice [Writer2xhtml] extension on one of my documents. The result is readable but not professional enough. There's no table of contents on the left. The code blocks lose their syntax highlighting.

Ultimately, I think the converter generated by Gemini is an alternative to what's suggested above. The result is comparable to that produced by Pandoc if the ODT document is suitable for the converter.

4. The initial prompt to Gemini 3

We assume here that the reader has some prior knowledge of [Gemini AI](#). If not, they are advised to read document [\[ref1\]](#). The initial prompt given to Gemini was in the form of the following text file [Prompt User.txt]:

1. User requests
- 2.
3. - Create a Python script [convert] to generate a MkDocs site using the Material theme from an ODT document. The name of the ODT document will be the sole argument for convert.
- 4.
5. - Then create a Python script [build] that will use the [mkdocs.yml] file created by convert to generate a static HTML site
6. - Finally, the [build] script will display the site's homepage in a browser.
- 7.
8. - The site will have two independent columns. The left column will contain the table of contents, and the right column will contain the content of the ODT document.
- 9.
10. - The table of contents will include all the headings from the ODT document (Heading 1, Heading 2, Heading 3, etc.), along with their numbering and hierarchy
- 11.
12. - Most of the images are in tables and have been resized.
13. You will search through the tables for these images and maintain their resized dimensions.
- 14.
15. - There are code blocks in the ODT document. They can be identified by their styles [Numbered Source Code Results, Source Code]
16. Other attributes of the code paragraph should be ignored. For example, if the code paragraph is also a numbered list, you should ignore this attribute.

17. All these code blocks must be treated as code on the MkDocs site. The lines of code must be numbered in the HTML output.
- 18.
19. - For all code blocks, you will use Python syntax highlighting.
- 20.
21. - You must not alter the lines of code. In particular, you must not remove the indentations or spaces preceding the first character of the line of code.
- 22.
23. - You must identify all bulleted lists. ODT has several ways to create them. You must explore them all. Bulleted lists are
24. sometimes nested within one another. You must preserve this nesting.
- 25.
26. - The title of the site will be "Generate a Python script with AI tools"
- 27.
28. - You will embed every image extracted from the ODT document as well as every MkDocs chapter created
- 29.
30. - You must preserve the links found in the ODT document. They must remain as links in the generated HTML document.
- 31.
32. - You must preserve text that is bold, underlined, or italicized. It must be rendered exactly as is in the HTML document
- 33.

- Line 19: The initial test was performed on a previously published course that contained Python code;
- this prompt was used to start the chat. After a few dozen iterations, it became obsolete;

5. Set up a Python development environment

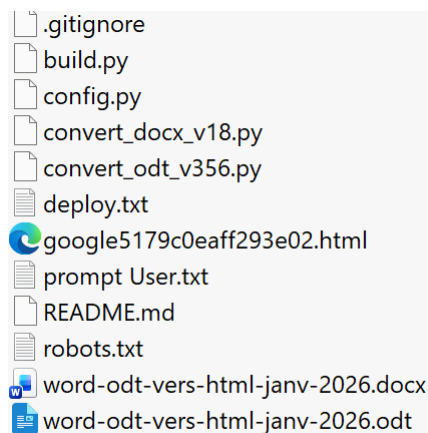
Gemini and ChatGPT will generate two Python scripts. We therefore need a Python environment:

- The installation of Python is described in [Section 3.1](#) of [\[ref1\]](#);
- The installation of PyCharm is described in [Section 3.2](#) of [\[ref1\]](#). [\[ref1\]](#) describes a lengthy PyCharm configuration. Here, PyCharm will only be used to run the Python scripts generated by Gemini and ChatGPT. Therefore, any PyCharm configuration is unnecessary. PyCharm will also be used to present the files required by the converter;

6. The converter's working directory

You can download the working directory |[HERE](#)|.

The working directory is as follows:



```

.gitignore
build.py
config.py
convert_docx_v18.py
convert_odt_v356.py
deploy.txt
google5179c0eaff293e02.html
prompt User.txt
README.md
robots.txt
word-odt-vers-html-janv-2026.docx
word-odt-vers-html-janv-2026.odt

```

- For now, we will ignore the following files [.gitignore, deploy.txt, google*.html, README.md, robots.txt, deploy.txt]. These are files that will be used to deploy the locally generated HTML site to GitHub;

- [prompt User.txt] is the prompt I used to start the iterations with Gemini;
- The file [word-odt-vers-html-janv-2026.odt] is the ODT document for this article. It is converted to HTML by the script [convert_odt_v356.py];
- The file [word-odt-to-html-jan-2026.docx] is the DOCX document for this same article. It is converted to HTML by the script [convert_docx_v18.py];
- The scripts [convert*.py] and [build.py] are the two Python scripts generated by the AI;
 - [convert*] converts an ODT or DOCX document into a [MkDocs] site. A MkDocs site is a static site whose pages are written using [Markdown] syntax. [MkDocs] provides a server capable of displaying MkDocs sites;
 - [build] converts the MkDocs site into a standard static HTML site. At the end of this conversion, it opens a browser to display the site's root directory;

At no point will we look at the generated Python code. We will treat both scripts as black boxes. At no point did I modify the code manually. I always asked Gemini / ChatGPT to correct its own script. That is why it is not necessary to know Python to use the converter.

- [config.py] is a configuration file for the [convert*] scripts. Initially, [convert*] did not have a configuration file. Then, as I switched between documents to convert, I gradually realized that there were elements I was asking the AI to generate (for example, the site name, which changes for each document) that would be better placed in a configuration file that the user could modify themselves. I built this file as well through iterations with the AI;

7. The converters' configuration file

The configuration file [config*.py] contains the information that can vary from one document to another. It begins as follows:

```

1. config = {
2.     # -----
3.     # MkDocs configuration
4.     # -----
5.     "mkdocs": {
6.         # Site title that will appear in the browser tab
7.         "site_name": "Convert a Word or ODT document to a static HTML site using Gemini 3 AI",
8.
9.         # Site publication URL (e.g., GitHub Pages)
10.        "site_url": "https://stahe.github.io/word-odt-vers-html-janv-2026/",
11.
12.        # Meta description for search engine optimization (SEO)
13.        "site_description": "Convert a Word or ODT document to a static HTML site using Gemini
3 AI",
14.
15.        # Site author
16.        "site_author": "Serge Tahé",
17.
18.        # Link to the source code repository (GitHub icon in the top right corner)
19.        "repo_url": "https://stahe.github.io/word-odt-vers-html-janv-2026",
20.        "repo_name": "GitHub",

```

- line 1: the configuration file is a Python script. It defines a single variable [config]. This is a dictionary that will contain all the configuration values;
- line 5: the [mkdocs] dictionary configures the [mkdocs.yml] file generated by the Gemini/ChatGPT converter. This file is then used by the [build] script to build an HTML site from the MkDocs site generated by the converter;
- Lines 5–20: configure the GitHub site that will host the HTML site created from the ODT/DOCX document (see paragraph 12)

The [config.py] configuration file continues as follows:

```

21.         # Important: set to False so that links work correctly on GitHub Pages
22.         # (prevents links like folder/ instead of folder/index.html)
23.         "use_directory_urls": False,

```

```

24.
25.     # Configuration of the Material theme for MkDocs
26.     "theme": {
27.         "name": "material",
28.         # Directory containing overrides (custom footer, etc.)
29.         "custom_dir": "overrides",
30.
31.         # Enabled navigation features
32.         "features": [
33.             "navigation.sections", # Groups sections in the menu
34.             "navigation.indexes", # Allows clicking on the section title
35.             "navigation.expand", # Opens menus by default
36.             "toc.integrate", # Integrates the table of contents on the left (if
supported)
37.             "navigation.top" # "Back to top" button
38.         ],
39.
40.         # Color palette (Light Mode / Dark Mode)
41.         "palette": [
42.             {
43.                 "media": "(prefers-color-scheme: light)",
44.                 "scheme": "default",
45.                 "primary": "teal",
46.                 "accent": "purple",
47.                 "toggle": {
48.                     "icon": "material/brightness-7",
49.                     "name": "Switch to dark mode"
50.                 }
51.             },
52.             {
53.                 "media": "(prefers-color-scheme: dark)",
54.                 "scheme": "slate",
55.                 "primary": "teal",
56.                 "accent": "purple",
57.                 "toggle": {
58.                     "icon": "material/brightness-4",
59.                     "name": "Switch to light mode"
60.                 }
61.             }
62.         ]
63.     },

```

- line 23: this line is important. If it is missing, instead of displaying an HTML page, the browser will open the folder containing that page;
- line 26: MkDocs offers several themes for a MkDocs/HTML site. Here we have chosen the “material” theme, line 27;
- lines 31–38: configuration of site navigation. This will be done using a table of contents located in the left column of the displayed page (line 36);
- lines 40–63: define two color palettes: light mode (lines 43–49) or dark mode (lines 52–61). An icon in the top bar of the displayed pages allows you to switch between them;

The rest of the configuration file is as follows:

```

64.     # Markdown extensions used to enrich the content
65.     "markdown_extensions": [
66.         "admonition", # Warning/info blocks
67.         "attr_list", # Custom CSS attributes {: .class }
68.         "pymdownx.superfences", # Advanced code blocks
69.         "pymdownx.mark", # Highlighting ==text==
70.         {
71.             "pymdownx.highlight": {
72.                 "anchor_linenums": True,
73.                 "linenums": None # Line numbers are managed dynamically by the script
74.             }
75.         },
76.         "md_in_html", # Allows Markdown within HTML tags (crucial for lists and tables)
77.         "footnotes" # Footnotes [^1]
78.     ],

```

- lines 64–78: extensions to the Markdown language used by MkDocs. These extensions were generated by Gemini following some of my requests;

The rest of the configuration file is as follows:

```
79.      # Additional scripts and CSS
80.      "extra_javascript": [
81.          "javascripts/focus.js" # Script for "Focus" mode (full screen)
82.      ],
83.      "extra_css": [
84.          "stylesheets/focus.css"
85.      ]
86.  },
```

- line 81: the [focus.js] script is a JavaScript script generated by Gemini. It is associated with the button on the site's top bar to show or hide the table of contents;
- line 84: the CSS used by this button;

The rest of the configuration file is as follows:

```
87.  # -----
88.  # Footer
89.  # -----
90.  "footer": (
91.      "{% block footer %}\n"
92.      "  <div class=\"md-footer-meta md-typeset\">\n"
93.      "    <div class=\"md-footer-meta__inner\">\n"
94.      "      \n"
95.      "      <a href=\"https://tahe.developpez.com\" target=\"_blank\">\n"
96.      "        https://tahe.developpez.com\n"
97.      "      >\n"
98.      "      \n"
99.      "      This tutorial, written by Serge Tahé, is made available to the public under
the terms of the\n"
100.      "        Creative Commons Attribution-NonCommercial-\n"
101.      "        ShareAlike 3.0 Unported License>.\n"
102.      "      \n\n"
103.      "    \n"
104.      "  "
105.      "{% endblock %}"
106.  ),
```

- lines 87–106: the definition of the HTML site footer. This was generated by Gemini based on a sample text I provided;

The rest of the configuration file is as follows:

```
107.  # -----
108.  # Additional configuration (Analytics, etc.)
109.  # -----
110.  "extra": {
111.      "analytics": {
112.          "provider": "google",
113.          "property": "G-XXXXXXX"
114.      }
115.  },
116.
117.  # -----
118.  # Document Title Detection
119.  # -----
120.  "document_title": {
121.      # ODT styles to be considered as the main document title (global H1)
122.      "style_names": [
123.          "p1"
124.      ],
125.      # CSS applied to this heading in the generated Markdown
126.      "css": "font-size: 28px; font-weight: bold; margin-bottom: 1em; line-height: 1.2;
color: #2c3e50;"
127.  },
```

- lines 110–115: definition of the Google Analytics (GA) tag that will track site visits;
- line 113: insert your GA tag here;

- lines 120-127: the style of the document title, the one present in the ODT/DOCX document before the first level 1 heading. The style specified on line 123 changes with each ODT document. However, it can be constant (“Title”) for DOCX documents. By default, the converter logs the styles of all paragraphs in the ODT/DOCX document preceding the first Level 1 heading. You must therefore run the converter once, identify the style of the paragraph you want to use as the title, and then set that style on line 123;
- line 126: the CSS style you want to apply to the document title. This title appears on the “Home” page, the first page displayed when the site opens;

The rest of the configuration file is as follows:

```

128.  # -----
129.  # Source Code Management
130.  # -----
131.  "code": {
132.      # Keyword in the ODT style to identify a code block
133.      "style_keywords": [
134.          "code"
135.      ],
136.      # Default language if no detection works
137.      "default_language": "text",
138.
139.      # Visual style of rich text (bold/italic/color preserved from ODT)
140.      "rich_line_height": "20px",
141.      "rich_font_family": "Consolas, 'Courier New', monospace",
142.      "rich_font_size": "15px",
143.
144.      # Rules for automatic language detection based on content
145.      "detection_rules": {
146.          "csharp": [
147.              "using", "Console.WriteLine", "public static void Main", "WebMethod",
148.              "TryParse", "EventArgs", "String.Format", "System.Web.Services"
149.          ],
150.          "java": [
151.              "System.out.println", "public static void main(String", "package",
152.              "JUnitTest", "Class.forName", "PreparedStatement", "private static void",
153.              "private void", "getAgendaMedecinJour", "@PostConstruct", "@ResponseBody",
154.              "@RequestMapping", "getDoctor", "@Entity", "@Autowired", "@Bean",
155.              "Serializable", "getClient", "getTimeSlot", "getAppointment",
156.              "PostAddAppointment", "PostDeleteAppointment", "@EnableJpaRepositories", "@Component",
157.              "getDoctorScheduleToday", "getResponse", "getMessagesForException", "getBase64", "addAppointment",
158.              "Response", "getPartialViewAgenda", "setModelForAgenda", "ActionContext",
159.              "getActionContext", "PostLang", "PostUser", "PostGetAgenda", "@NotNull",
160.              "@EnableAutoConfiguration", "HttpSecurity"
161.          ],
162.          "html": [
163.              "<html>", "<body>", "<script>", "href=", "<span>", "<html>",
164.              "<form>", "<table>", "<input"
165.          ],
166.          "sql": [
167.              "SELECT", "INSERT INTO", "UPDATE", "DELETE FROM", "WHERE",
168.              "CREATE TABLE", "ALTER TABLE"
169.          ],
170.          "python": [
171.              "def", "import", "print(", "from"
172.          ],
173.          "xml": [
174.              "<?xml", "<project", "<version>", "<configuration>", "<build>",
175.              "<dependency>", "<properties>", "<configuration>", "<start-class>"
176.          ],
177.          "javascript": [
178.              "use strict", "console.log", "let", "constructor", "async", "export"
179.          ],
180.          "php": [
181.              "<?php", "declare", "require"
182.          ],
183.          "vbscript": [
184.              "Option", "Dim", "Explicit"
185.          ]
186.      }
187.  },

```

- lines 128–187: This configuration is used for "plain text" code that does not initially have syntax highlighting. These lines are intended to apply syntax highlighting based on the language used in the code block. **Note that if all your code is already formatted because it comes from an IDE such as Eclipse, Visual Studio, etc., and you do not have any "plain text" code blocks associated with a language, then you do not need to enter anything in this section of the configuration. You can leave the tables associated with the languages blank.**
- Lines 133–135: Code blocks are identified by their styles in the ODT/DOCX document. There may be several. On line 134, list all the keywords that allow a code style to be detected. In my documents, all my code styles have the word "code" in their names. And no other styles have this word in their names. So I can simply enter a single keyword. If there are several, separate them with commas on line 134;
- Lines 145–187: the strings that identify the language of a code block. Some keywords may appear in multiple languages. The converter selects the language for which it found the most keywords. The principle is simple: you look at your code, select characteristic keywords, and enter them in these lines for the correct language. Note that you cannot enter just anything as a language name: you must use the names recognized by MkDocs;
- lines 139–142: these lines concern formatted code blocks (bold, italics, underlining, highlighting, character color). These blocks are not processed like "plain text" code blocks. They are rendered exactly as written in the HTML;
 - line 140: sets the line height for code lines;
 - Line 141: Sets the CSS font for the code block;
 - line 142: sets the font size;
- line 137: sets the default language. In the case of a "plain text" code block, if no language is detected, the default language will be "text." For MkDocs, this means a code block without syntax highlighting. This is the case, for example, with execution results;

The configuration ends as follows:

```

188.  # -----
189.  # Static files to copy to the site root
190.  # -----
191.  "files_to_copy": [
192.      "googleXXXXXX.html",
193.      "robots.txt",
194.      "word-odt-to-html-jan-2026.pdf"
195.  ],
196.
197.  # Debug mode: displays the styles encountered before the first heading
198.  "debug": True
199. }
```

- lines 191–195: the list of files to copy to the root of the HTML site;
 - lines 192–193: the site's Google Analytics tag and the [robots.txt] file that allows search engines to crawl the site;
 - line 194: the PDF of this document made available for download;
- line 198: [debug] set to True enables logging of the styles of the paragraphs preceding the first Level 1 heading in the ODT/DOCX document. It is in these logs that we will find the exact style of the paragraph to be used as the title on the site's homepage;

Ultimately, when switching from one ODT/DOCX document to another, you will need to modify lines [6-20, 127-129, 201] and certain lines in the language key strings. The rest remains unchanged.

Other elements have been added to the configuration file in the latest versions of the converter:

[Copy] button

On rich text code blocks as well as on "plain text" code blocks containing a recognized language, there is a [Copy] button:

Java

```
1 package istia.st.spring.core;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.context.support.ClassPathXmlApplicationContext;
8
9 public class Demo01 {
10
```

Copier



This button allows you to copy the code block to the clipboard. Line numbers are not included. The behavior of this button is controlled by the following lines in the [config.py] configuration file:

```
1.      # -----
2.      # [Copy] button in code blocks (copies to the clipboard)
3.      # -----
4.
5.      # Enables/disables the [Copy] button (True = enabled)
6.      "copy_button": True,
7.
8.      # Button label before copying
9.      "copy_label": "Copy",
10.
11.     # Temporary label after a successful copy
12.     "copy_copied_label": "Copied",
13.
14.     # If True, the button only appears if the block's language is "recognized"
15.     # (e.g., language-java / language-python, etc.)
16.     "copy_only_recognized_language": True,
17.
18.     # Minimum number of lines to display the button:
19.     # - 0 => no threshold (all "recognized language" blocks will have the button)
20.     # - >0 => the block must have at least this number of lines
21.     "copy_min_lines": 0,
22.
23.     # If True, the button is also allowed on "colored" Pygments blocks
24.     # even if the language-xxx class is not explicitly present.
25.     # Useful for certain code renderings (highlighttable) where coloring exists
26.     # but the language does not appear in a CSS class.
27.     "copy_allow_pygments_heuristic": True,
28.
29.     # Appearance of the [Copy] button (inline CSS: only declarations,
30.     # without curly braces). Allows for easy customization of the look.
31.     # -----
32.     # "Material-like" style for the [Copy] button
33.     # (inline CSS: declarations only, no curly braces)
34.     # -----
35.     "copy_style": {
36.         # Code block container: required to position the button
37.         "container": "position: relative;",
38.
39.         # Normal button style
40.         "btn": (
41.             "position: absolute; top: .5rem; right: .5rem; "
42.             "display: inline-flex; align-items: center; justify-content: center; "
43.             "gap: .35rem; "
44.             "padding: 0.25rem 0.6rem; "
45.             "font-size: .72rem; font-weight: 600; letter-spacing: .01em; "
46.             "line-height: 1.2; "
47.             "border-radius: 999px; "
48.             "border: 1px solid rgba(0,150,136,0.45); "
49.             "background: rgba(0, 150, 136, 0.12); "
50.             "color: rgb(0,150,136); "
51.             "box-shadow: 0 1px 2px rgba(0,0,0,.10); "
52.             "backdrop-filter: saturate(180%) blur(6px); "
53.             "cursor: pointer; user-select: none; "
54.             "transition: transform .08s ease, box-shadow .12s ease, background .12s ease; "
```

```

55.         "z-index:5;"
56.     ),
57.
58.     # Hover: slightly "raised" (like Material)
59.     "btn_hover": (
60.         "background:rgba(0,150,136,.20); "
61.         "box-shadow:0 3px 10px rgba(0,0,0,.18); "
62.         "transform:translateY(-1px);"
63.     ),
64.
65.     # "Copied" state: slight fade
66.     "btn_copied": "opacity:.85;"
67. },

```

Images

Similarly, the behavior of clickable images is controlled by the following lines in the configuration file:

```

1.     # -----
2.     # Images: shadow settings (zoom / lightbox)
3.     # -----
4.     "images": {
5.         "shadow": {
6.             # Enables/disables the shadow around images
7.             "enabled": True,
8.
9.             # Radius applied to image corners (e.g., "0px", "4px", "8px", ...)
10.            "border_radius": "8px",
11.
12.            # Default shadow on "zoomable" images
13.            "zoomable": "0 8px 24px rgba(0,0,0,.28), 0 18px 60px rgba(0,0,0,.20)",
14.
15.            # Hover shadow on "zoomable" images
16.            "zoomable_hover": "0 12px 30px rgba(0,0,0,.32), 0 26px 80px rgba(0,0,0,.22)",
17.
18.            # Shadow when the image is displayed in full size (lightbox)
19.            "lightbox": "0 24px 80px rgba(0,0,0,.65)",
20.        }
21.    },

```

Language recognition for unformatted code blocks has been improved. In [config.py], the following lines have been added to the [code] dictionary:

```

1.     "code": {
2.
3.         # case-insensitive languages
4.         "case_insensitive_languages": [
5.             "sql", "html", "vbnet", "vbscript"
6.         ],

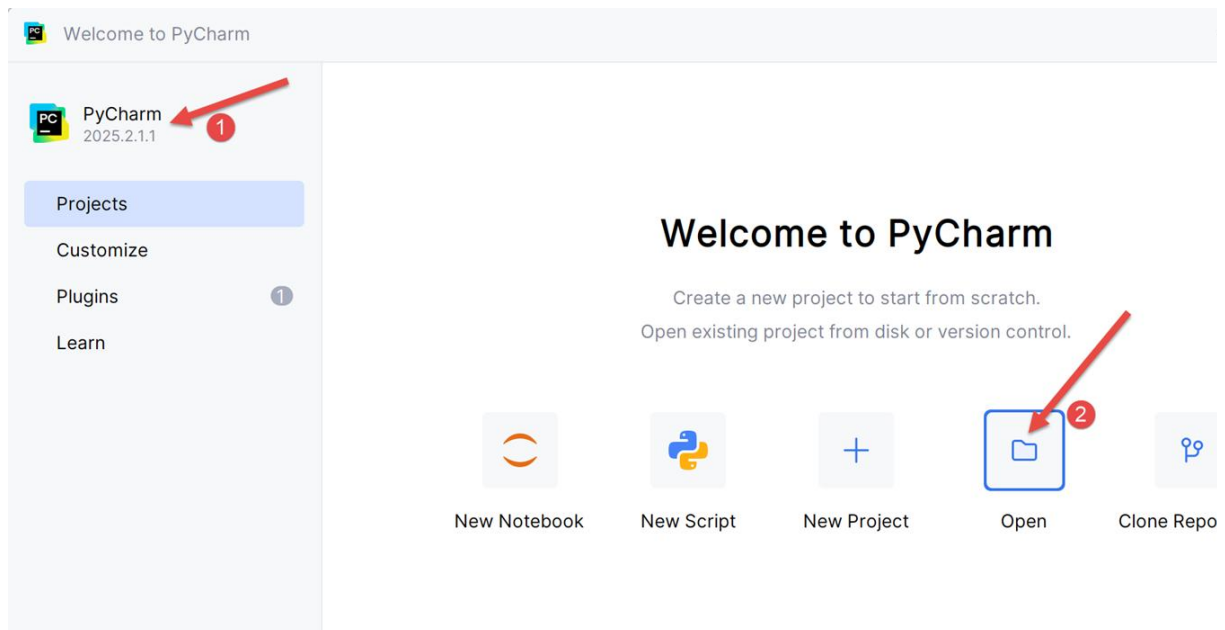
```

- Lines 3–6 specify case-insensitive languages (upper/lowercase). Therefore, in the key expressions associated with the HTML language, there is no need to include both <body> and <BODY>, as both are valid HTML tags. The <body> tag is sufficient.

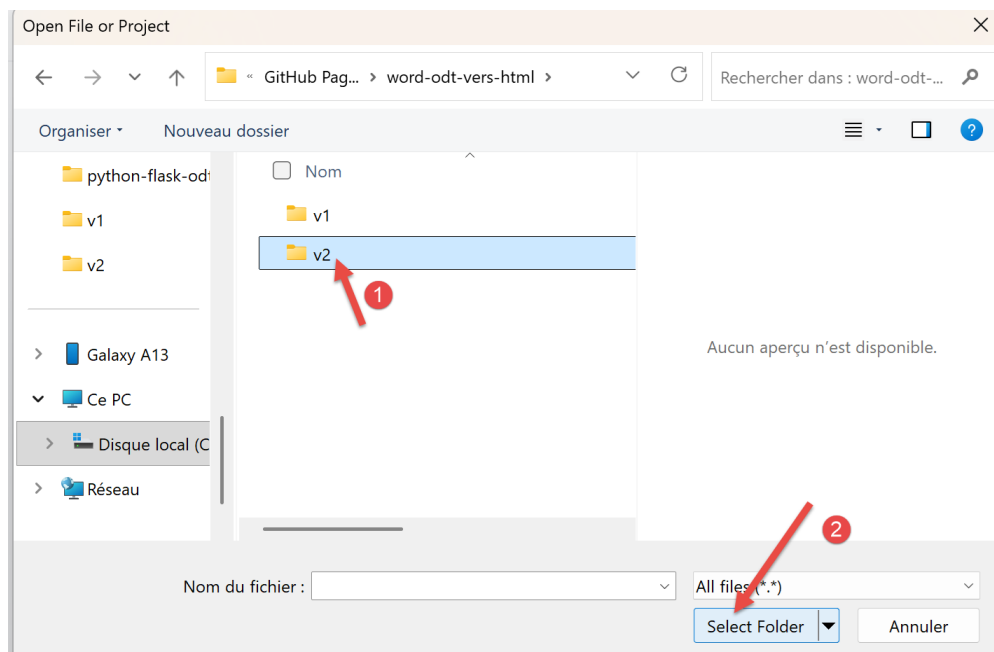
8. Using the ODT → HTML Converter

Note that the working directory can be found |[HERE](#)|.

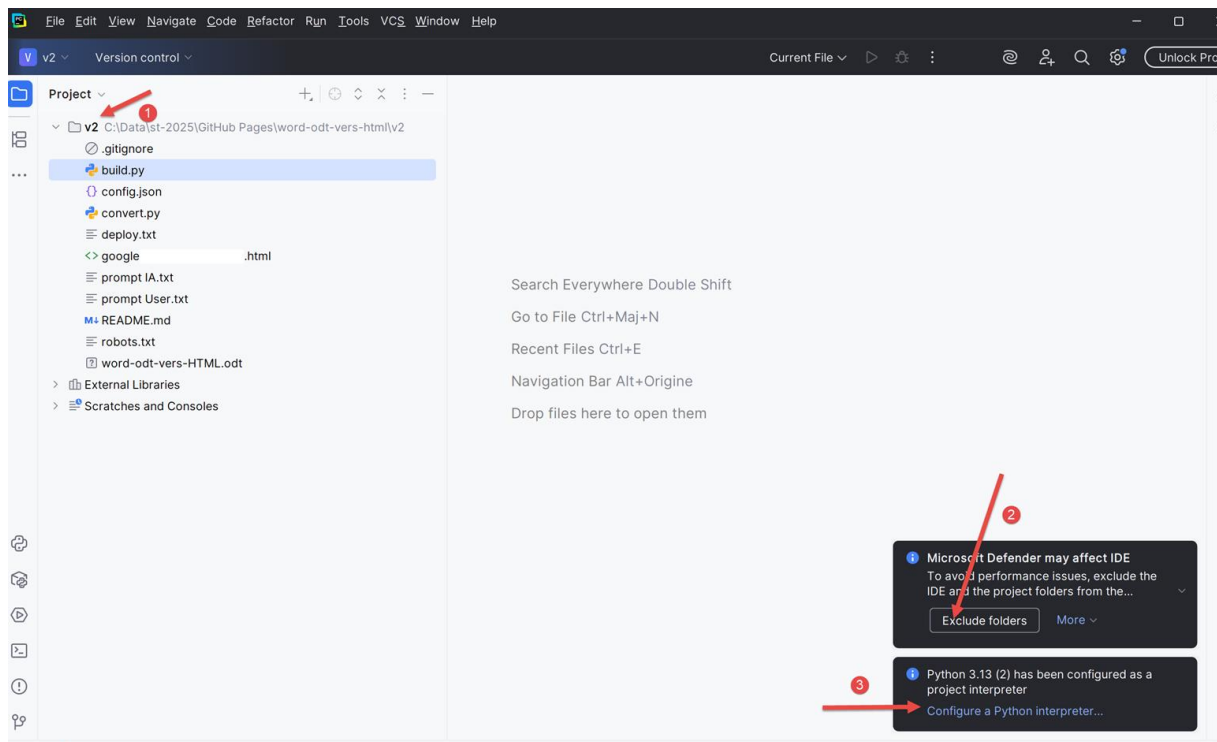
Let's launch PyCharm:



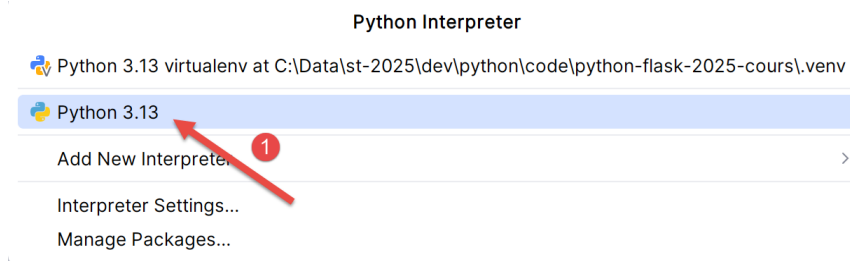
- In [2], open a new project;



- In [1], select the folder where you saved the working directory;
- In [2], confirm your selection;



- In [1], select your Python project;
- In [2], confirm to avoid being bothered by the Windows security system;
- In [3], choose a Python interpreter;



- In [1], select the Python interpreter you installed in step 5;

The Gemini/ChatGPT converter uses Python dependencies that you need to install. We'll ask Gemini which ones:

rappelle moi toutes les dépendances Python nécessaires à ton convertisseur [convert]

Gemini's response is as follows:

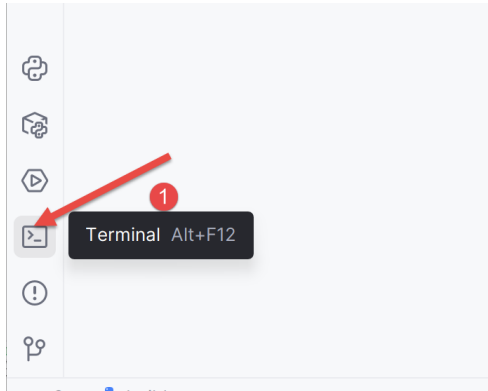
Commande d'installation rapide

Vous pouvez installer tout le nécessaire en une seule commande dans votre terminal :

Bash

```
pip install odfpv unicode mkdocs mkdocs-material
```

We'll install these dependencies:



- In [1], open a terminal;



- In [2], type the command provided by Gemini;

The response is as follows:

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> pip install odfpv unicode mkdocs
mkdocs-material
2. Defaulting to user installation because normal site-packages is not writable
3. Requirement already satisfied: odfpv in c:\users\serge\appdata\roaming\python\python313\site-
packages (1.4.1)
4. Requirement already satisfied: unicode in
c:\users\serge\appdata\roaming\python\python313\site-packages (1.4.0)
5. Requirement already satisfied: mkdocs in c:\users\serge\appdata\roaming\python\python313\site-
packages (1.6.1)
6. Requirement already satisfied: mkdocs-material in
c:\users\serge\appdata\roaming\python\python313\site-packages (9.7.0)
7. Requirement already satisfied: defusedxml in
c:\users\serge\appdata\roaming\python\python313\site-packages (from odfpv) (0.7.1)
8. Requirement already satisfied: click>=7.0 in
c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (8.3.1)
9. Requirement already satisfied: colorama>=0.4 in
c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (0.4.6)
```

```

10. Requirement already satisfied: ghp-import>=1.0 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (2.1.0)
11. Requirement already satisfied: jinja2>=2.11.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (3.1.6)
12. Requirement already satisfied: markdown>=3.3.6 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (3.10)
13. Requirement already satisfied: markupsafe>=2.0.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (3.0.2)
14. Requirement already satisfied: mergedeep>=1.3.4 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (1.3.4)
15. Requirement already satisfied: mkdocs-get-deps>=0.2.0 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (0.2.0)
16. Requirement already satisfied: packaging>=20.5 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (25.0)
17. Requirement already satisfied: pathspec>=0.11.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (0.12.1)
18. Requirement already satisfied: pyyaml-env-tag>=0.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (1.1)
19. Requirement already satisfied: pyyaml>=5.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (6.0.2)
20. Requirement already satisfied: watchdog>=2.0 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (6.0.0)
21. Requirement already satisfied: babel>=2.10 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (2.17.0)
22. Requirement already satisfied: backrefs>=5.7.post1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (6.1)
23. Requirement already satisfied: mkdocs-material-extensions>=1.3 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (1.3.1)
24. Requirement already satisfied: paginate>=0.5 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (0.5.7)
25. Requirement already satisfied: pygments>=2.16 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (2.19.2)
26. Requirement already satisfied: pymdown-extensions>=10.2 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (10.17.2)
27. Requirement already satisfied: requests>=2.26 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (2.32.5)
28. Requirement already satisfied: python-dateutil>=2.8.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from ghp-import>=1.0->mkdocs)
   (2.9.0.post0)
29. Requirement already satisfied: platformdirs>=2.2.0 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-get-
   deps>=0.2.0->mkdocs) (4.5.0)
30. Requirement already satisfied: six>=1.5 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from python-
   dateutil>=2.8.1->ghp-import>=1.0->mkdocs) (1.17.0)
31. Requirement already satisfied: charset_normalizer<4,>=2 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-
   material) (3.4.3)
32. Requirement already satisfied: idna<4,>=2.5 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-
   material) (3.10)
33. Requirement already satisfied: urllib3<3,>=1.21.1 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-
   material) (2.5.0)
34. Requirement already satisfied: certifi>=2017.4.17 in
   c:\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-
   material) (2025.8.3)
35. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2>

```

On my PC, everything was already installed. If that's not the case for you, all the required dependencies will be installed.

Now we can use the converter. In the open terminal, type the following command:

```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> python .\convert_odt_v356.py .\word-odt-
   to-html-jan-2026.odt .\config.py
2. --- ODT to MkDocs Converter V356 ---
3. Processing .\word-odt-to-html-jan-2026.odt...
4. Copied: google5179c0eaff293e02.html
5. Copied: robots.txt
6. Copied: word-odt-to-html-jan-2026.pdf
7. Copied: word-odt-to-html-jan-2026.zip

```

```

8. Scanning references...
9. Generating Markdown...
10. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
11. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
12. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
13. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
14. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
15. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
16. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
17. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
18. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
19. [DEBUG PRE-H1] Tag=text:p Style='Standard (WW)' Text='...'
20. [DEBUG PRE-H1] Tag=text:p Style='Standard (WW)' Text='...'
21. [DEBUG PRE-H1] Tag=text:p Style='P1' Text='Convert a Word or ODT document to a sta...'
22. >>> DOCUMENT TITLE FOUND: Convert a Word or ODT document to a MkDocs-compatible static HTML
    site with Gemini 3 and ChatGPT 5.2 AI
23. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
24. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
25. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
26. [DEBUG PRE-H1] Tag=text:p Style='P3' Text='Serge Tahé, January 2026...'
27. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
28. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
29. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
30. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='...'
31. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='...'
32. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='...'
33. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='This site was created with the [Word or ...]'
34. [DEBUG PRE-H1] Tag=text:h Style='P5' Text='Introduction...'
35. >>> CHAPTER 1: 1 Introduction
36. >>> CHAPTER 1: 2 Examples in this document
37. >>> CHAPTER 2: 2.1 Lists
38. >>> CHAPTER 3: 2.1.1 Bulleted lists
39. >>> CHAPTER 3: 2.1.2 Numbered lists
40. >>> CHAPTER 3: 2.1.3 Mixed lists 1
41. >>> CHAPTER 3: 2.1.4 Mixed lists 2
42. >>> CHAPTER 2: 2.2 Code Blocks
43. >>> CHAPTER 3: 2.2.1 Rich text code blocks (Eclipse, Visual Studio, ...)
44. >>> CHAPTER 3: 2.2.2 Plain text code blocks
45. >>> CHAPTER 2: 2.3 Links
46. >>> CHAPTER 2: 2.4 Text Formatting
47. >>> CHAPTER 2: 2.5 A title can also be formatted.
48. >>> CHAPTER 2: 2.6 Images
49. >>> CHAPTER 2: 2.7 Special characters
50. >>> CHAPTER 2: 2.8 Tables
51. >>> CHAPTER 2: 2.9 Footnotes
52. >>> CHAPTER 1: 3 What's on the Internet
53. >>> CHAPTER 1: 4 The Initial Prompt for Gemini 3
54. >>> CHAPTER 1: 5 Setting up a Python development environment
55. >>> CHAPTER 1: 6 The converter's working directory
56. >>> CHAPTER 1: 7 The converter configuration file
57. >>> CHAPTER 1: 8 Using the ODT → HTML converter
58. >>> CHAPTER 1: 9 Using the DOCX → HTML converter
59. >>> CHAPTER 1: 10 Building the Static HTML Site
60. >>> CHAPTER 1: 11 Reviewing the Generated HTML Site
61. >>> CHAPTER 2: 11.1 The Site's Top Bar
62. >>> CHAPTER 2: 11.2 The Site Footer
63. >>> CHAPTER 2: 11.3 The Home Page
64. >>> CHAPTER 2: 11.4 Bulleted Lists
65. >>> CHAPTER 2: 11.5 Numbered lists
66. >>> CHAPTER 3: 11.5.1 Mixed lists 1
67. >>> CHAPTER 3: 11.5.2 Mixed Lists 2
68. >>> CHAPTER 2: 11.6 Rich Code Blocks
69. >>> CHAPTER 3: 11.6.1 Example 1
70. >>> CHAPTER 3: 11.6.2 Example 2
71. >>> CHAPTER 3: 11.6.3 Example 3
72. >>> CHAPTER 2: 11.7 Plain Text Blocks
73. >>> CHAPTER 3: 11.7.1 Example 1
74. >>> CHAPTER 3: 11.7.2 Example 2
75. >>> CHAPTER 3: 11.7.3 Example 3
76. >>> CHAPTER 2: 11.8 Other code blocks
77. >>> CHAPTER 2: 11.9 Links
78. >>> CHAPTER 2: 11.10 Text Formatting
79. >>> CHAPTER 2: 11.11 Images
80. >>> CHAPTER 2: 11.12 Special characters

```

```

81. >>> CHAPTER 2: 11.13 Tables
82. >>> CHAPTER 2: 11.14 Footnotes
83. >>> CHAPTER 2: 11.15 Known Issues
84. >>> CHAPTER 2: 11.16 Other Cases
85. >>> CHAPTER 1: 12 Hosting the HTML Site on GitHub
86. >>> CHAPTER 1: 13 Tracking the Site with Google Analytics and Google Search Console
87. >>> CHAPTER 1: 14 Conclusion
88. Done.

```

- Line 1: the command that converts the ODT document into a MkDocs site [**python convert_odt_v356.py .\word-odt-to-html-jan-2026.odt .\config.py**]. Adjust the version number (here 356) to the version you downloaded. The first parameter of the converter is the ODT document to be converted, the second is the converter's configuration file;
- lines 4–7: the files that the converter copies to the root of the MkDocs site it creates;
- lines 10–34: debugging the styles of the paragraphs preceding the first Level 1 heading. These paragraphs will form the home page. One of the paragraphs serves as the title of the home page and thus of the site. This is the paragraph on line 21. Note its style, P1. You must include this style in the configuration file:

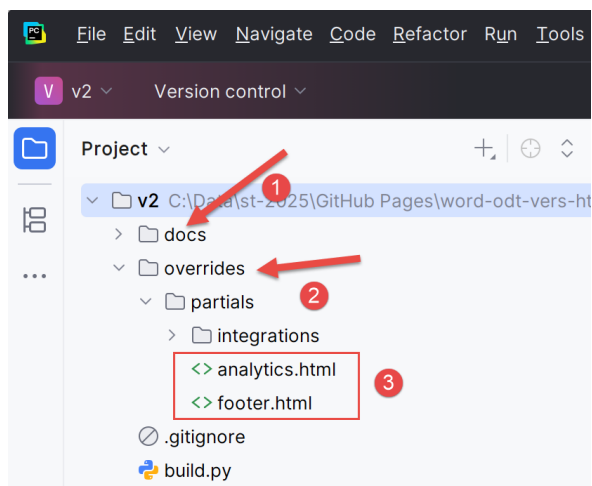
```

7.      # -----
8.      # Document Title Detection
9.      # -----
10.     "document_title": {
11.         # ODT styles to be considered as the main document title (global H1)
12.         "style_names": [
13.             "P1"
14.         ],
15.         # CSS applied to this heading in the generated Markdown
16.         "css": "font-size: 28px; font-weight: bold; margin-bottom: 1em; line-height: 1.2;
17.             color: #2c3e50;"

```

- lines 35-88: I asked the AI to log all the chapters it encountered;

This execution has modified your working directory:



- In [1], [docs] is the MkDocs site that the Gemini/ChatGPT converter created. You might be curious to visit it;
- in [2], an [overrides] folder has been created. It will be used by the [build] tool for the HTML site;
- in [3]: [analytics.html] will be used for tracking the site via Google Analytics. [footer.html] is the footer you defined in the [config.py] file;

We could use the MkDocs site right now. The command [**python -m mkdocs serve**] lets you view it. You can try:

```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python -m mkdocs serve
2. INFO - Building documentation...
3. INFO - Cleaning site directory

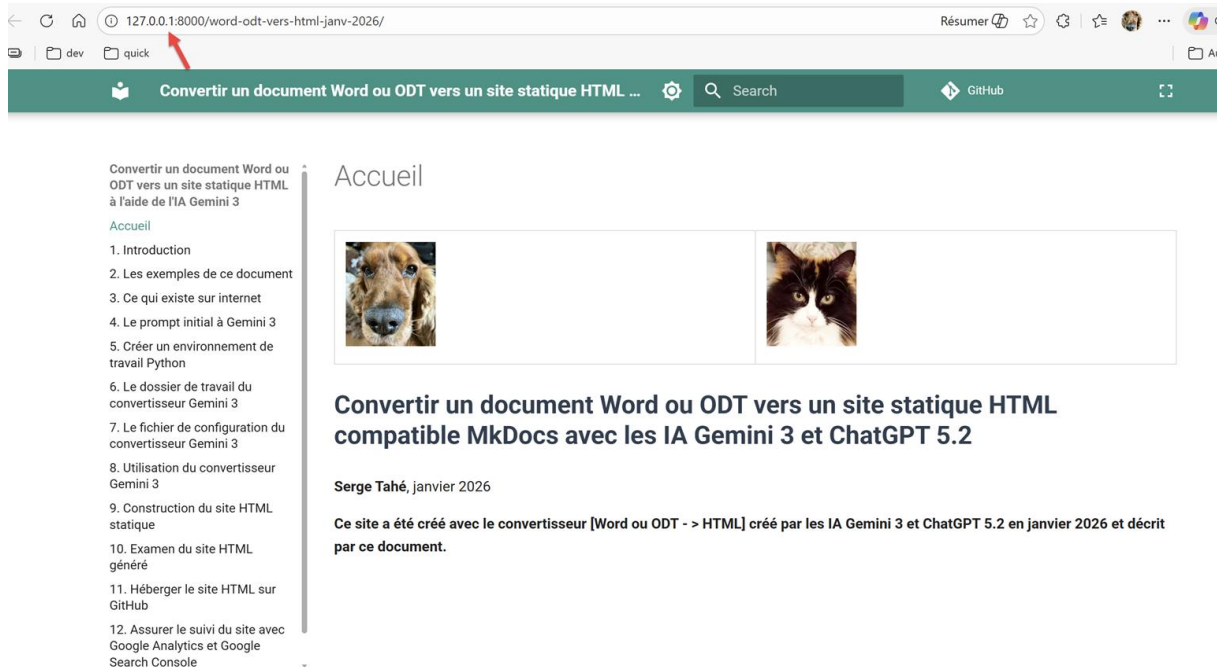
```

```

4. INFO - Doc file 'les-exemples.md' contains a link '#_Les_exemples', but there is no such
      anchor on this page.
5. INFO - Documentation built in 0.41 seconds
6. INFO - [15:46:06] Serving on http://127.0.0.1:8000/word-odt-vers-html-janv-2026/

```

Ctrl-click on the link on line 6. The MkDocs site should appear:



Here is the result of a long effort. To stop the MkDocs server, simply press Ctrl-C in the terminal that launched it.

9. Using the DOCX → HTML Converter

The command for converting a Word DOCX document is very similar to that for converting a LibreOffice ODT document. We will modify the document title style in [config.py]:

```

1.  # -----
2.  # Document Title Detection
3.  # -----
4.  "document_title": {
5.      # ODT styles to be considered as the main document title (global H1)
6.      "style_names": [
7.          "Title"
8.      ],
9.      # CSS applied to this title in the generated Markdown
10.     "css": "font-size: 28px; font-weight: bold; margin-bottom: 1em; line-height: 1.2;
11.     color: #2c3e50;"
12. },

```

- Line 7: Enter "Title". This is the style of the DOCX document you are going to convert. We'll see this in the converter's debug logs.

Still in the PyCharm terminal, type the following command:

```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> python .\convert_docx_v18.py .\word-odt-
   to-html-jan-2026.docx .\config.py
2. C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2\convert_docx_v18.py:976: SyntaxWarning:
   invalid escape sequence '\h'
3. - REF Bookmark \h
4. --- DOCX to MkDocs Converter V16 ---
5. Copied: google5179c0eaff293e02.html

```

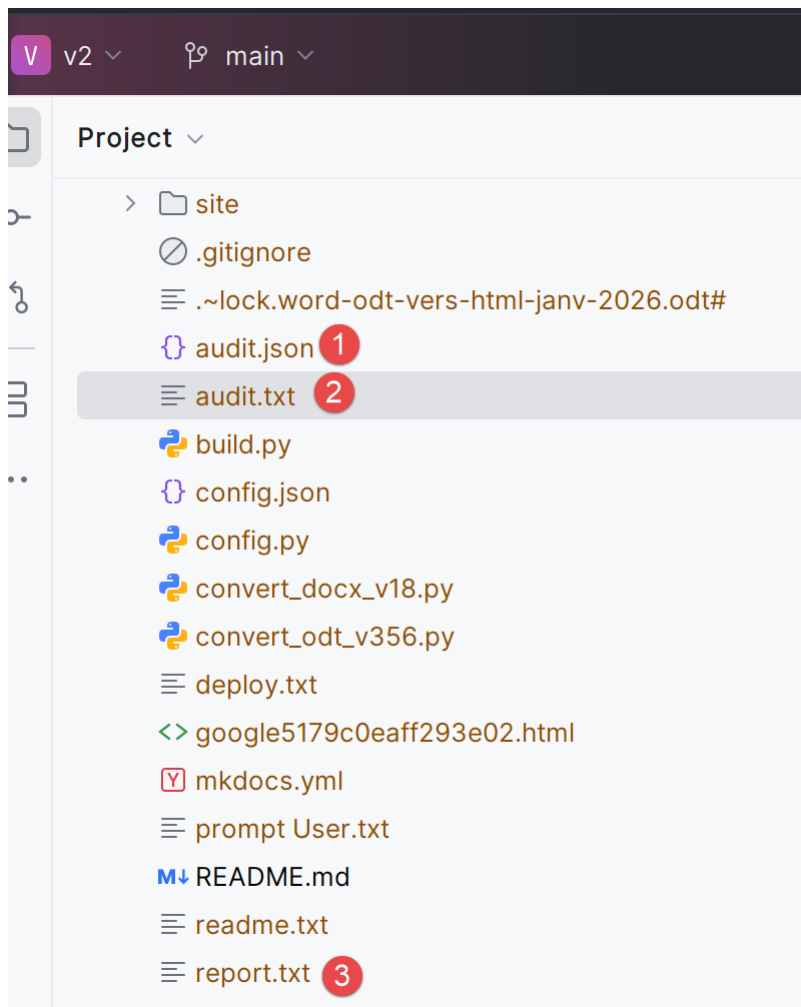


```

6. Copied: robots.txt
7. Copied: word-odt-to-html-jan-2026.pdf
8. Copied: word-odt-to-html-jan-2026.zip
9. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
10. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
11. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
12. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
13. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
14. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
15. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
16. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
17. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
18. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
19. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
20. [DEBUG PRE-H1] style=Title heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='<span>Convert a Word or ODT document to a Mk-compatible static HTML site...'
21. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
22. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
23. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
24. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='<b>Serge Tahé</b><span>, January 2026</span>...'
25. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
26. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
27. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
28. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
29. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
30. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='...'
31. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None
txt='<B>This site was created using the 'Word or ODT - &gt; HTML' converter...'
32. [DEBUG PRE-H1] style=Title1 heading=1 rebase=0 numId=1 ilvl=0 list=numPr:1/ordered
txt='<span>Introduction</span>...'
33. Done. (audit.json, audit.txt, report.txt generated)

```

- line 1: the command is as follows: `[python .\convert_docx_v18.py .\word-odt-to-html-jan-2026.docx .\config.py]`. (Adjust the version number (here 18) to the version you downloaded.):
 - the first parameter `[\convert_docx_v18.py]` is the DOCX → HTML converter
 - the second parameter `[\word-odt-to-html-jan-2026.docx]` is the name of the DOCX document to be converted;
 - the third parameter `[\config.py]` is the configuration file;
- Line 33: The converter reports that three files have been generated:



The [audit.txt] file is as follows:

```
1. Version: V16
2. Paragraphs: 2029
3. Tables: 97
4. Images (blips): 2
5. Headings detected (raw): 53
6. Minimum heading level detected (raw): 1
7. Rebase offset applied: 0
8.
9. Top paragraph styles:
10. - SourceCodenumrot: 1054
11. - StandardWW: 594
12. - Standard: 146
13. - Paragraphedeliste: 113
14. - SourceCodenumRotResults: 33
15. - new code: 28
16. - Title 2: 25
17. - Heading 1: 14
18. - Title 3: 14
19. - StandardWWW: 6
20. - Plain text: 1
21. - Heading: 1
22.
23. List paragraphs:
24. - with numbering: 1329
25. - by style fallback: 49
26. - Not recognized: 0
```

- line 2: the number of paragraphs in the Word document;
- line 3: the number of tables;

- lines 9–21: the styles found in the document;
 - lines 10, 14, 15: the style of the code blocks. A single style would probably have sufficed;
 - lines 11–12, 19: the style of standard paragraphs. A single style would probably have sufficed;
 - lines 16–18, 21: the styles of the document's headings. On line 21, we see that only one paragraph has the "Heading" style. This is the document title that precedes the first "Heading 1";

This audit of the Word document is a good way to assess the document's quality. Here I can see that I used too many different styles for the same thing in my Word document.

The [audit.json] file is identical to the [audit.txt] file but in JSON format:

```

1.  {
2.    "version": "V16",
3.    "file": "word-odt-to-html-jan-2026.docx",
4.    "counts": {
5.      "paragraphs": 2029,
6.      "tables": 97,
7.      "image_blips": 2,
8.      "headings_raw": 53
9.    },
10.   "lists": {
11.     "with_numpr": 1329,
12.     "by_style": 49,
13.     "unrecognized": 0
14.   },
15.   "heading": {
16.     "min_level_raw": 1,
17.     "rebase_offset": 0
18.   },
19.   "top_styles": [
20.     [
21.       "SourceCodenumrot",
22.       1054
23.     ],
24.     [
25.       "StandardWW",
26.       594
27.     ],
28.     [
29.       "Standard",
30.       146
31.     ],
32.     [
33.       "ListParagraph",
34.       113
35.     ],
36.     [
37.       "SourceCodeNumberResults",
38.       33
39.     ],
40.     [
41.       "newcode",
42.       28
43.     ],
44.     [
45.       "Title2",
46.       25
47.     ],
48.     [
49.       "Title1",
50.       14
51.     ],
52.     [
53.       "Title3",
54.       14
55.     ],
56.     [
57.       "StandardWWW",
58.       6
59.     ],
60.     [
61.       "Plain text",

```

```

62.     1
63.     ],
64.     [
65.         "Title",
66.         1
67.     ]
68. ]
69. }

```

The [report.txt] file is as follows:

```

1. [SUMMARY] Lists detected via "by style" fallback (aggregated)
2.   - Paragraphedeliste -> level=1 type=unordered: 49
3.
4. [SUMMARY] Ignored Word blocks (aggregated)
5.   - <w:sectPr>: 1

```

I didn't understand that...

It is possible to request only an audit of the Word document to assess its quality using the [--audit] parameter:

```
python .\convert_docx_v18.py .\word-odt-to-html-jan-2026.docx .\config.py --audit
```

In this case, only the document audit is performed. The MkDocs site is not generated.

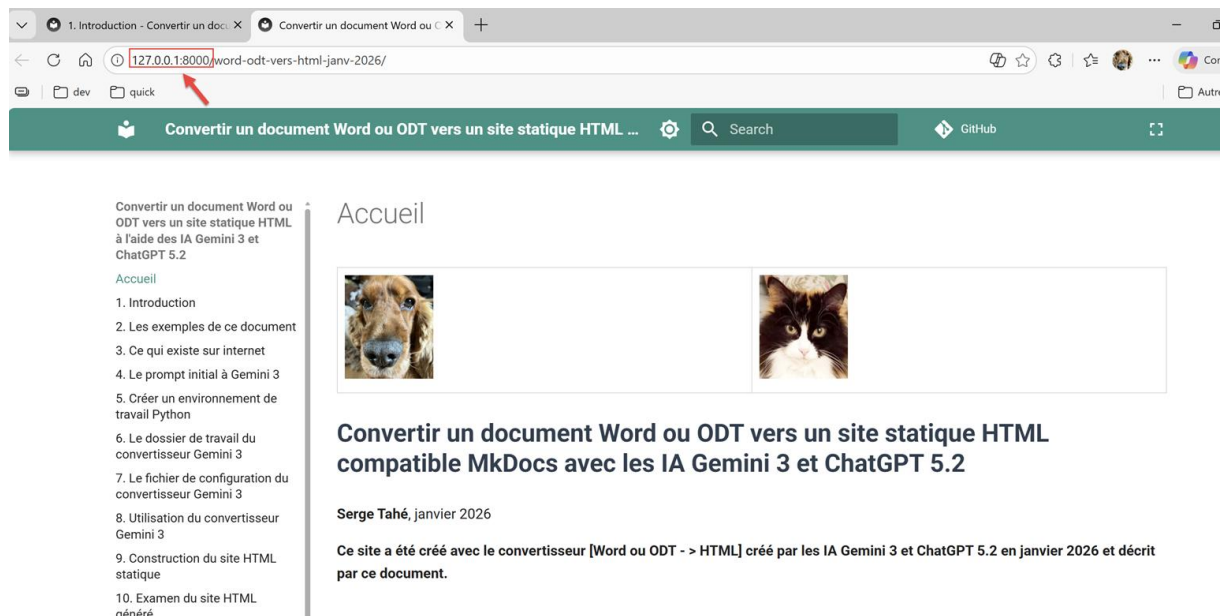
As shown earlier, you can view the MkDocs site generated by the converter:

```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> python -m mkdocs serve
2. INFO      - Building documentation...
3. INFO      - Cleaning site directory
4. INFO      - Documentation built in 0.59 seconds
5. INFO      - [06:05:48] Serving on http://127.0.0.1:8000/word-odt-vers-html-janv-2026/

```

Ctrl-Click on the URL in line 5:



10. Building the static HTML site

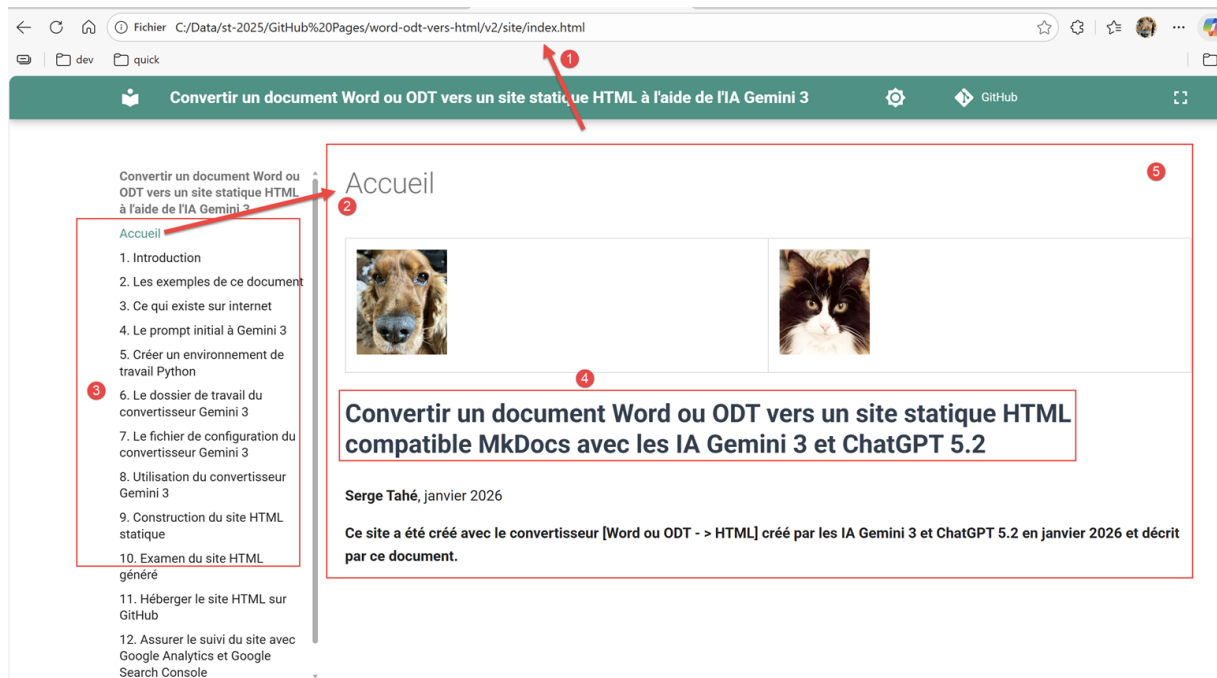
From now on, there are no longer any differences between ODT and DOCX documents. We work on the MkDocs site built by either converter.

Now let's build the HTML site. This is done using the [build] script. Still in the terminal, type the following command:

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> python .\build.py
2. Starting to build the MkDocs site...
3. Build successful! The site is located at: C:\Data\st-2025\GitHub Pages\word-odt-vers-
html\v2\site
4. Opening C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\site\index.html in the browser...
```

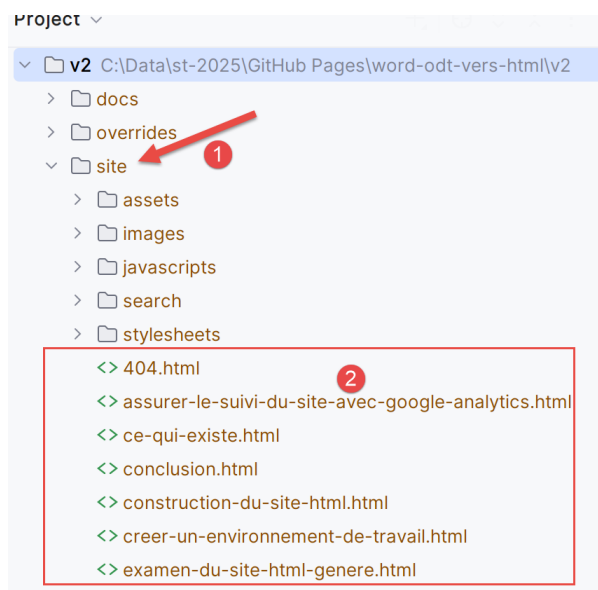
- line 1: the [python build] command creates the HTML site from the MkDocs site;
- line 4: the site is displayed in a browser;

The result is the same as for the MkDocs site:



- In [1], we can see that an HTML page is being displayed;
- In [2], this page corresponds to the site's home page;
- In [3], the table of contents for the HTML document;
- In [4], the document title has been formatted according to its configuration in [config.py];
- In [5], the first page displayed is [Home]. The content of this page is the content of the ODT/DOCX document **preceding the first Chapter 1-level chapter**;

Running [build] has modified your working directory:



- In [1], a new [site] folder has been created. It contains your static HTML site;
- In [2], the pages of your site;

11. Reviewing the generated HTML site

We will now examine the HTML output of this ODT/DOCX document. We have already seen that the converter preserves the table of contents.

11.1. The top bar of the site

Let's look at the site's top bar:



- In [1], the site name defined in [config.py];
- At [2], the icon that lets you switch between dark and light modes;
- At [3], the icon that links to the GitHub repository where the HTML site will be exported. Also defined in [config.py];
- At [4], the icon that lets you hide or show the table of contents;

11.2. The site's footer

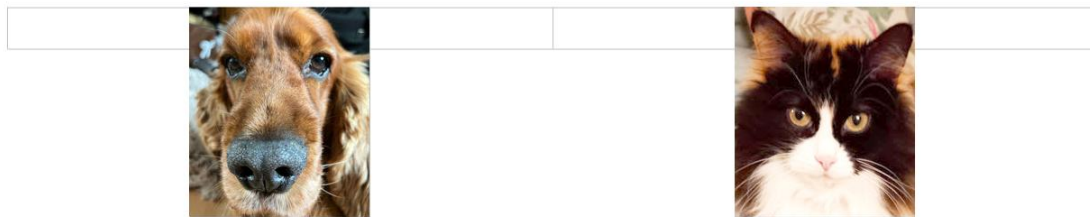
Now let's look at the footer:



This is the footer defined in the [config.py] file.

11.3. The home page

The title page of the ODT/DOCX document was as follows:



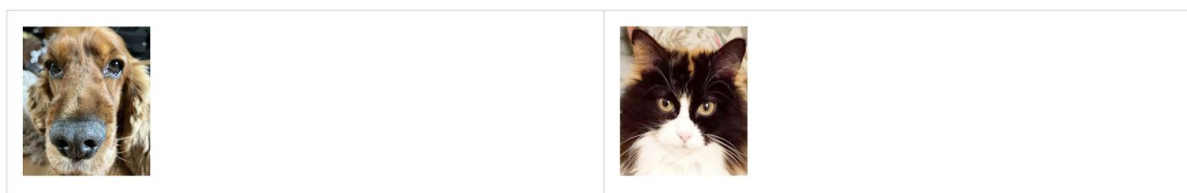
Convertir un document Word ou ODT vers un site statique HTML compatible MkDocs avec les IA Gemini 3 et ChatGPT 5.2

Serge Tahé, janvier 2026

Ce site a été créé avec le convertisseur [Word ou ODT - > HTML] créé par les IA Gemini 3 et ChatGPT 5.2 en janvier 2026 et décrit par ce document.

This title page of the ODT/DOCX document becomes the HTML site's home page:

Accueil



**Convertir un document Word ou ODT vers un site statique HTML
compatible MkDocs avec les IA Gemini 3 et ChatGPT 5.2**

1

Serge Tahé, janvier 2026

Ce site a été créé avec le convertisseur [Word ou ODT - > HTML] créé par les IA Gemini 3 et ChatGPT 5.2 en janvier 2026 et décrit par ce document.

The Gemini 3 / ChatGPT converter places everything above—in the ODT / DOCX document, the first Level 1 heading, i.e., the “Title 1” style—on the Home page. If you include images as shown above, it will display them. You can therefore imagine giving your site a cover, just like a real book. In [1], this is the main title of the document. Its rendering is controlled by the following lines in the configuration file [config.py]:

```
1. # -----
2. # Document Title Detection
3. # -----
4. "document_title": {
5.     # ODT styles to be considered as the main title of the document (global H1)
6.     "style_names": [
7.         "P1"
8.     ],
9.     # CSS applied to this title in the generated Markdown
10.    "css": "font-size: 28px; font-weight: bold; margin-bottom: 1em; line-height: 1.2;
11.    color: #2c3e50;"
12. },
```

- lines [6-8]: the list of possible styles for your document’s title. When I look at this document, the LibreOffice style for my title is ‘Main Title’. However, the Gemini converter couldn’t find it. It logged the styles it encountered, and this displayed [P1]. This is a major issue with LibreOffice: the displayed style names do not match the internal names used by the software. They are only there to adapt to the user’s language;
- Line 10: Once the main title has been detected, you can configure its rendering. I wanted a font size of 28 (font-size: 28px;) and bold (font-weight: bold);

With the images and the title style, you can create an attractive home page.

The main title of your document may not have one of the styles defined in lines [6-8]. To find the style of your main title, use the following line in the [config.py] file

```
1. "debug": True
```

With the value [true], the style of the paragraphs preceding the first Level 1 heading—i.e., the paragraphs on the homepage—will be displayed when the Gemini/ChatGPT converter runs. So, for a document other than this one, I obtained the following logs:

```
1. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
2. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
3. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
4. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
5. [DEBUG PRE-H1] Style='P3' (Clean='p3') | Text='...'
6. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
7. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
8. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
9. [DEBUG PRE-H1] Style='P1' (Clean='p1') | Text='...'
10. [DEBUG PRE-H1] Style='P4' (Clean='p4') | Text='Introduction to PHP7 through examples...'
11. >>> DETECTED DOCUMENT TITLE: Introduction to the PHP7 language through examples
12. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
13. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
14. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
15. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
16. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
17. [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Text='...'
18. [DEBUG PRE-H1] Style='P5' (Clean='p5') | Text='Serge Tahé, July 2019...'
19. [DEBUG PRE-H1] Style='P5' (Clean='p5') | Text='...'
20. [DEBUG PRE-H1] Style='P6' (Clean='p6') | Text='...'
21. [DEBUG PRE-H1] Style='Heading 1' (Clean='heading 1') | Text='Introduction to PHP 7...'
```

- line 10, the document title has the ‘P4’ style;

In the [config.py] file, I then added the following lines:

```
1. "document_title": {
2.     "style_names": [
3.         "P4"
4.     ],
5.     "css": "font-size: 28px; font-weight: bold; margin-bottom: 1em; line-height: 1.2; color:
6.     #2c3e50;"
7. },
```

- line 3, the style I was looking for;

This is why the debugger displays the lines:

```
1. [DEBUG PRE-H1] Style='P4' (Clean='p4') | Text='Introduction to the PHP7 language through
   examples...'
2. >>> DOCUMENT TITLE DETECTED: Introduction to the PHP7 language through examples
```

It encountered the 'P4' style and then displays that the document title has been found. Once it has been found, you can set the [debug] key to [false] in [config.py]:

```
1. "debug": False
```

Now let's look at the conversion of the [Examples] chapter, which contains the examples that the Gemini/ChatGPT converter can handle:

11.4. Bulleted lists

ODT / DOCX document:

Listes à puces

- Élément 1 ;
- Élément 2 :
- Élément 3 ;
 - Élément 3.1 ;
 - Élément 3.1.1
 - Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

Above, the text [Bulleted lists] is highlighted because it is a reference associated with a cross-reference.

HTML document:

Listes à puces

- Élément 1 ;
- Élément 2 :
- Élément 3 ;
 - Élément 3.1 ;
 - Élément 3.1.1
 - Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

Note that while the ODT/DOCX document uses different bullet styles, the HTML document uses only one type of bullet.

11.5. Numbered lists

ODT/DOCX document:

Listes numérotées

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 1. Élément 2.1.1.1
 2. Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

HTML document:

2.1.2. Listes numérotées

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 1. Élément 2.1.1.1
 2. Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

11.5.1. Mixed lists 1

ODT/DOCX document

2.1.3. Listes mixtes 1

- Élément 1 ;
- Élément 2 ;
- Élément 3 ;
 - Élément 3.1 ;
 1. Élément 3.1.1
 2. Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

HTML document

2.1.3. Listes mixtes 1

- Élément 1 ;
- Élément 2 :
- Élément 3 ;
 - Élément 3.1 ;
 1. Élément 3.1.1
 2. Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

Here too, there are sometimes differences between the types of bullets used.

11.5.2. Mixed lists 2

ODT / DOCX document

2.1.4. Listes mixtes 2

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 - Élément 2.1.1.1
 - Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

HTML document

2.1.4. Listes mixtes 2

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 - Élément 2.1.1.1
 - Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

11.6. Rich code blocks

Rich code blocks are rendered identically in HTML (except for the background color). Here are three examples:

11.6.1. Example 1

ODT / DOCX file

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplica
8.
9. public class Demo01 {
10.
11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // récupération du contexte Spring
14.         ApplicationContext ctx = new ClassPathXmlAppli
15.         // on récupère les beans
16.         Personne p01 = ctx.getBean("personne_01", Perso
17.         Personne p02 = ctx.getBean("personne_02", Perso
18.         List<Personne> club = ctx.getBean("club", new
19.         ArrayList<Personne>().getClass());
20.         Appartement appart01 = ctx.getBean(Appartement
```

HTML output

Java

```
1 package istia.st.spring.core;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.context.support.ClassPathXmlApplicat
8
9 public class Demo01 {
10
11     @SuppressWarnings({ "unchecked", "resource" })
12     public static void main(String[] args) {
13         // récupération du contexte Spring
14         ApplicationContext ctx = new ClassPathXmlApplicationCor
15         // on récupère les beans
16         Personne p01 = ctx.getBean("personne_01", Personne.class);
17         Personne p02 = ctx.getBean("personne_02", Personne.class);
18         List<Personne> club = ctx.getBean("club", new ArrayList<
19         Appartement appart01 = ctx.getBean(Appartement.class);
```

11.6.2. Example 2

ODT / DOCX document:

Python

```
1. # -----
2. def affiche(chaine):
3.     # affiche chaine
4.     print("chaine=%s" % chaine)
5.
6.
7. # -----
8. def affiche_type(variable):
9.     # affiche le type de variable
10.    print("type[%s]=%s" % (variable, type(variable)))
11.
12.
13. # -----
14. def f1(param):
15.     # ajoute 10 à param
16.     return param + 10
17.
18.
19. # -----
20. def f2():
21.     # rend un tuple de 3 valeurs
22.     return "un", 0, 100
23.
24.
25. # -----
26. ...
```

HTML output

Python

```
1 # -----
2 def affiche(chaine):
3     # affiche chaine
4     print("chaine=%s" % chaine)
5
6
7 # -----
8 def affiche_type(variable):
9     # affiche le type de variable
10    print("type[%s]=%s" % (variable, type(variable)))
11
12
13 # -----
14 def f1(param):
15     # ajoute 10 à param
16     return param + 10
17
18
19 # -----
20 def f2():
21     # rend un tuple de 3 valeurs
22     return "un", 0, 100
```

11.6.3. Example 3

ODT / DOCX document

ECMAScript

```
1. 'use strict';
2. // ceci est un commentaire
3. // constante
4. const nom = "dupont";
5. // un affichage écran
6. console.log("nom : ", nom);
7. // un tableau avec des éléments de type différents
8. const tableau = ["un", "deux", 3, 4];
9. // son nombre d'éléments
10. let n = tableau.length;
11. // une boucle
12. for (let i = 0; i < n; i++) {
13.     console.log("tableau[" + i + "] = ", tableau[i]);
14. }
15. // initialisation de 2 variables avec le contenu
16. let [chaine1, chaine2] = ["chaine1", "chaine2"];
17. // concaténation des 2 chaînes
18. const chaine3 = chaine1 + chaine2;
19. // affichage résultat
20. console.log([chaine1, chaine2, chaine3]);
21. ...
```

HTML output

ECMAScript

```
1  'use strict';
2  // ceci est un commentaire
3  // constante
4  const nom = "dupont";
5  // un affichage écran
6  console.log("nom : ", nom);
7  // un tableau avec des éléments de type différent
8  const tableau = ["un", "deux", 3, 4];
9  // son nombre d'éléments
10 let n = tableau.length;
11 // une boucle
12 for (let i = 0; i < n; i++) {
13     console.log("tableau[" + i + "] = ", tableau[i]);
14 }
15 // initialisation de 2 variables avec le contenu d
16 let [chaine1, chaine2] = ["chaine1", "chaine2"];
17 // concaténation des 2 chaînes
18 const chaine3 = chaine1 + chaine2;
19 // affichage résultat
20 console.log([chaine1, chaine2, chaine3]);
21 ...
```

11.7. Plain text code blocks

Plain text code blocks found in the ODT / DOCX document are syntax-highlighted by MkDocs based on the language found in the code block. To help the converter identify the correct language, "key strings" have been added to the [config.py] file for each language. The converter counts the "key strings" found. It then associates the code block with the language that has the most "key strings" found.

Let's look at a few examples.

11.7.1. Example 1

ODT / DOCX document (Java)

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplicationContext;
8.
9. public class Demo01 {
10.
11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // récupération du contexte Spring
14.         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15.         // on récupère les beans
16.         Personne p01 = ctx.getBean("personne_01", Personne.class);
17.         Personne p02 = ctx.getBean("personne_02", Personne.class);
18.         List<Personne> club = ctx.getBean("club", new ArrayList<Personne>().getClass());
19.         Appartement appart01 = ctx.getBean(Appartement.class);
20.         ...
    }
```

HTML Output

Java

```
1 package istia.st.spring.core;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.context.support.ClassPathXmlApplicationContext;
8
9 public class Demo01 {
10
11     @SuppressWarnings({ "unchecked", "resource" })
12     public static void main(String[] args) {
13         // récupération du contexte Spring
14         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15         // on récupère les beans
16         Personne p01 = ctx.getBean("personne_01", Personne.class);
17         Personne p02 = ctx.getBean("personne_02", Personne.class);
18         List<Personne> club = ctx.getBean("club", new ArrayList<Personne>().getClass());
19         Appartement appart01 = ctx.getBean(Appartement.class);
20         ...
    }
```

In the HTML document, we can see that the Java code has been syntax-highlighted.

11.7.2. Example 2

ODT / DOCX document (XML)


```

1.    <?xml version="1.0" encoding="utf-8" ?>
2.    <configuration>
3.
4.        <configSections>
5.            <sectionGroup name="spring">
6.                <section name="context" type="Spring.Context.Support.ContextSupport" />
7.                <section name="objects" type="Spring.Context.Support.DefaultObjectFactory" />
8.            </sectionGroup>
9.        </configSections>
10.
11.        <spring>
12.            <context>
13.                <resource uri="config://spring/objects" />
14.            </context>
15.            <objects xmlns="http://www.springframework.net">
16.                <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
17.                    <constructor-arg index="0" value="MySQL.Data.MySqlClient" />
18.                    <constructor-arg index="1"
19.                    value="Server=localhost;Database=bdimpots;Uid=adminimpots;Pwd=mdpimpots;" />
20.                    <constructor-arg index="2" value="select limite, coeffr, co" />
21.                </object>
22.                <object name="metier" type="Metier.ImpotMetier, ImpotsV7-metier">
23.                    <constructor-arg index="0" ref="dao" />
24.                </object>
25.            </objects>
26.        </spring>
27.    </configuration>

```

HTML output

XML

```

1  <?xml version="1.0" encoding="utf-8" ?>
2  <configuration>
3
4      <configSections>
5          <sectionGroup name="spring">
6              <section name="context" type="Spring.Context.Support.ContextSupport" />
7              <section name="objects" type="Spring.Context.Support.DefaultObjectFactory" />
8          </sectionGroup>
9      </configSections>
10
11      <spring>
12          <context>
13              <resource uri="config://spring/objects" />
14          </context>
15          <objects xmlns="http://www.springframework.net">
16              <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
17                  <constructor-arg index="0" value="MySQL.Data.MySqlClient" />
18                  <constructor-arg index="1" value="Server=localhost;Database=bdimpots;Uid=adminimpots;Pwd=mdpimpots;" />
19                  <constructor-arg index="2" value="select limite, coeffr, co" />
20              </object>
21              <object name="metier" type="Metier.ImpotMetier, ImpotsV7-metier">
22                  <constructor-arg index="0" ref="dao" />
23              </object>
24          </objects>
25      </spring>
26  </configuration>

```

11.7.3. Example 3

ODT / DOCX document (HTML)

HTML

```
1. <!DOCTYPE HTML>
2. <HTML>
3.   <head>
4.     <title>Laragon</title>
5.
6.     <link href="https://fonts.googleapis.com/css?
   type="text/css">
7.
8.     <style>
9.       HTML, body {
10.        height: 100%;
11.      }
12.
13.      body {
14.        margin: 0;
15.        padding: 0;
16.        width: 100%;
17.        display: table;
18.        font-weight: 100;
19.        font-family: 'Karla';
20.      }
21.
22.      .container {
23.        text-align: center;
24.        display: table-cell;
25.        vertical-align: middle;
26.      }
27.
28.      .content {
29.        text-align: center;
30.        display: inline-block;
31.      }
```

HTML output

HTML

```
1 <!DOCTYPE HTML>
2 <HTML>
3   <head>
4     <title>Laragon</title>
5
6     <link href="<a href="view-source:https://fonts
7
8     <style>
9       HTML, body {
10        height: 100%;
11      }
12
13      body {
14        margin: 0;
15        padding: 0;
16        width: 100%;
17        display: table;
18        font-weight: 100;
19        font-family: 'Karla';
20      }
21
22      .container {
23        text-align: center;
24        display: table-cell;
25        vertical-align: middle;
26      }
27
```

11.8. Other code blocks

ODT / DOCX document

An execution result with a first line that does not start at 1:

Résultats d'exécution

On notera que le code ne commence pas avec la ligne n° 1.

```
7. C:\Data\st-2020\dev\python\cours-2020\python3-flask-2020\ve
C:/Data/st-2020/dev/python/cours-2020/python3-flask-2020/ba
8. nom=dupont
9. liste[0]=un
10. liste[1]=deux
11. liste[2]=3
12. liste[3]=4
13. [chaine1,chaine2,chaine1chaine2]
14. chaine=chaine1
15. type[4]=<class 'int'>
16. type[chaine1]=<class 'str'>
17. type[['un', 'deux', 3, 4]]=<class 'list'>
18. type[a changé]=<class 'str'>
```

HTML output

Résultats d'exécution

On notera que le code ne commence pas avec la ligne n° 1.

```
7 C:\Data\st-2020\dev\python\cours-2020\python3-flask-2
8 nom=dupont
9 liste[0]=un
10 liste[1]=deux
11 liste[2]=3
12 liste[3]=4
13 [chaine1,chaine2,chaine1chaine2]
14 chaine=chaine1
15 type[4]=<class 'int'>
16 type[chaine1]=<class 'str'>
17 type[['un', 'deux', 3, 4]]=<class 'list'>
18 type[a changé]=<class 'str'>
```

A code block that is not numbered in ODT remains so in the HTML:

ODT / DOCX document

Le convertisseur sait gérer les blocs de code non numérotés.

```
liste[0]=un
liste[1]=0
liste[2]=100
liste[0]=8
liste[1]=5
somme=13
```

HTML rendering

Le convertisseur sait gérer les blocs de code non numérotés.

```
liste[0]=un
liste[1]=0
liste[2]=100
liste[0]=8
liste[1]=5
somme=13
```

11.9. Links

ODT / DOCX document

2.3. Les liens

Le convertisseur Gemini sait conserver les liens externes du document ODT. Par exemple [Gemini 3](#) ou [\[Générer un script Python avec des outils d'IA\]](#).

Il sait gérer un lien vers un chapitre [Lien vers un chapitre](#)

Un renvoi vers un chapitre : [2.1](#).

Un renvoi vers un repère de texte qui précède : [Gemini 3](#)

Un renvoi vers un repère de texte qui suit : [GitHub](#)

HTML rendering

2.3. Les liens

Le convertisseur Gemini sait conserver les liens externes du document ODT. Par exemple [Gemini 3](#) ou [\[Générer un script Python avec des outils d'IA\]](#).

Il sait gérer un lien vers un chapitre [Lien vers un chapitre](#)

Un renvoi vers un chapitre : [2.1](#).

Un renvoi vers un repère de texte qui précède : [Gemini 3](#)

Un renvoi vers un repère de texte qui suit : [GitHub](#)

11.10. Text formatting

ODT / DOCX document

Un texte avec des mots en **gras**, en *italiques*, soulignés ou **surlignés** ou **surlignés** ou **surlignés**.

C'est vrai également pour les liens : [Générer un script Python avec des outils d'IA].

Le **convertisseur** gère également la **couleur** des **caractères**.

Il gère également les bordures supérieure et inférieure des paragraphes.

2.5. Un **titre** peut être également *enrichi*.

HTML rendering

2.4. L'enrichissement de texte

Le convertisseur sait gérer le gras, l'italique, le souligné et le surlignage. Il respecte la couleur du surlignage.

Un texte avec des mots en **gras**, en *italiques*, soulignés ou surlignés ou surlignés ou surlignés.

C'est vrai également pour les liens : [Générer un script Python avec des outils d'IA].

Le **convertisseur** gère également la **couleur** des **caractères**.

Il gère également les bordures supérieure et inférieure des paragraphes.

2.5. Un **titre** peut être également *enrichi*.

11.11. Images

2.5. Les images

Le convertisseur Gemini sait gérer les images et les tableaux d'images :



Note that the Gemini / ChatGPT converter preserves image resizing done in the ODT / DOCX document.

11.12. Protected characters

ODT / DOCX document

L'astérisque * a une signification Markdown. La ligne suivante peut être alors mal interprétée :

L'impôt I est alors égal à $0.15 \cdot R - 2072.5 \cdot \text{nbParts}$.

HTML Rendering

L'astérisque * a une signification Markdown. La ligne suivante peut être alors mal interprétée :

L'impôt I est alors égal à $0.15 \cdot R - 2072.5 \cdot \text{nbParts}$.

ODT / DOCX document (Markdown)

```
5.      ---
6.
7.      ## 📄 Description
8.
9.      On se propose dans ce projet de mettre à disposition du
documents Word ou ODT vers un site statique HTML.
10.
11.     Lorsque le document ODT convient, le convertisseur prod
l'aspect professionnel des sites produits par Pandoc.
12.
13.     ## 📄 Contexte de création
14.
15.     Ce convertisseur a été entièrement construit par l'IA *
est le résultat d'itérations successives pour gérer finement la
(OpenDocument Text).
16.
17.     ## ✨ Fonctionnalités
18.
19.     Le script `convert.py` effectue les actions suivantes :
20.
21.     * **Conversion ODT vers Markdown** : Analyse le fichier
structure.
22.     * **Gestion des Titres** : Génère automatiquement la Ta
latérale.
23.     * **Blocs de Code** : Détection automatique des langage
précise de la numérotation des lignes** (attributs `start-value
24.     * **Listes** : Support des listes à puces et numérotées
25.     * **Mise en forme** : Support du *gras*, *italique*, *s
respect des couleurs d'origine).
```

HTML output

Un autre exemple est lorsque vous voulez insérer un bloc de code MarkDown dans votre document

```
1 # Convertisseur Word/ODT vers Site HTML (MkDocs)
2
3  **[Voir le site de démonstration généré](https://stahe.github.io/word-odt-v
4
5 ---
6
7 ##  Description
8
9 On se propose dans ce projet de mettre à disposition du lecteur un convertisse
10
11 Lorsque le document ODT convient, le convertisseur produit un site HTML via **
12
13 ##  Contexte de création
14
15 Ce convertisseur a été entièrement construit par l'IA **Gemini 3** (avec un abo
16
17 ##  Fonctionnalités
18
19 Le script `convert.py` effectue les actions suivantes :
20
```

- Markdown code has been preserved;

11.13. Tables

ODT / DOCX document

1	2		<pre>1. package istia.st.spring.core; 2. 3. import java.util.ArrayList; 4. import java.util.List; 5. 6. import org.springframework.context.ApplicationContext; 7. import org.springframework.context.support.ClassPathXmlApplication Context; 8. 9. public class Demo01 { 10. 11.</pre>
3	4		<pre>1. # ----- 2. def affiche(chaine): 3. # affiche chaine 4. print("chaine=%s" % chaine) 5. 6. 7. # ----- 8. def affiche_type(variable): 9. # affiche le type de variable 10. print("type[%s]=%s" % (variable, type(variable))) 11. 12.</pre>

HTML rendering

Un tableau peut contenir différents contenus :

1	2		<pre>1 package istia.st.spring.core; 2 3 import java.util.ArrayList; 4 import java.util.List; 5 6 import org.springframework.context.ApplicationContext; 7 import org.springframework.context.support.ClassPathXmlApplicationConte 8 9 public class Demo01 { 10 11</pre>
3	4		<pre>1 # ----- 2 def affiche(chaine): 3 # affiche chaine 4 print("chaine=%s" % chaine) 5</pre>

11.14. Footnotes

Le convertisseur Gemini² gère les notes de bas de page. Voici une autre note³ de bas de page.

1. MkDocs
2. Google Gemini
3. La note de bas de page

11.15. Known issues

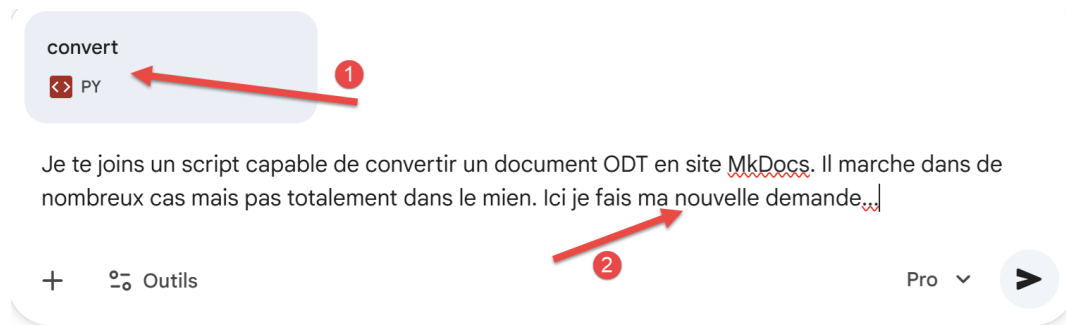
Some issues have been identified, but they can be corrected by modifying the ODT / DOCX:

- **code blocks must be followed by a blank line;** otherwise, the code block may render incorrectly. The identified issue is code immediately followed by a heading without being separated from it by a blank line;
- Bulleted lists cannot have a bottom border. **To add one, insert a blank line after the last list item;**
- **Bulleted lists must be nested.** Thus, a level 2 list must always be included within a level 1 list; otherwise, the level 2 list is rendered as code;

As new versions of the converter are released, some of these issues will be resolved. The three mentioned above can be avoided by correcting the source document.

11.16. Other Cases

If your document uses features other than those mentioned above, it is highly likely that these will not be supported by the Gemini/ChatGPT converter. What should you do then? You can submit your new request to one of the AIs by providing it with the current converter:



- In [1], I attach the converter for this document;
- In [2], I make my new request;

You'll likely go through many iterations. When a version is stable, make a note of its number so you can provide it to the AI in case of a regression. It's also a good idea to make a copy of each stable version. A major drawback of both AIs is that they regress quite easily. All it takes is asking for a new feature for the AI to break code that was previously working. Hence the importance of noting the version numbers of stable versions so you can revert to them. In January 2026, it seemed to me that ChatGPT 5.2 was less tend to regress than Gemini 3.

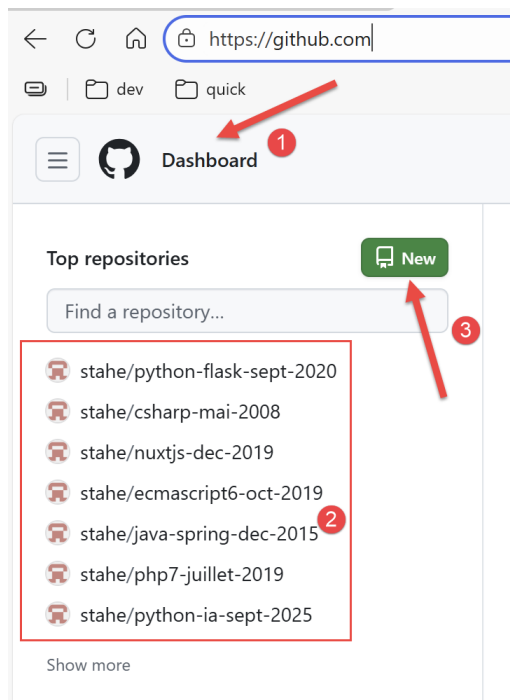
12. Hosting the HTML site on GitHub

It was Gemini itself that suggested I host the HTML site generated by its two scripts on GitHub. I didn't know that was possible. GitHub⁴ is a site that hosts development projects. Exporting programming courses there seems like a natural fit.

First, you need to have a GitHub account. If necessary, create one.

Log in to your GitHub account:

⁴ Footnote for GitHub



- In [2], your existing repositories, if you have any;
- In [3], create a new GitHub repository;

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner * stahe / Repository name * 1

✔ word-odt-vers-html-janv-2026 is available.

Great repository names are short and memorable. How about [sturdy-telegram](#)?

Description

2

84 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license

Licenses explain how others can use your code. [About licenses](#)

3 Create repository

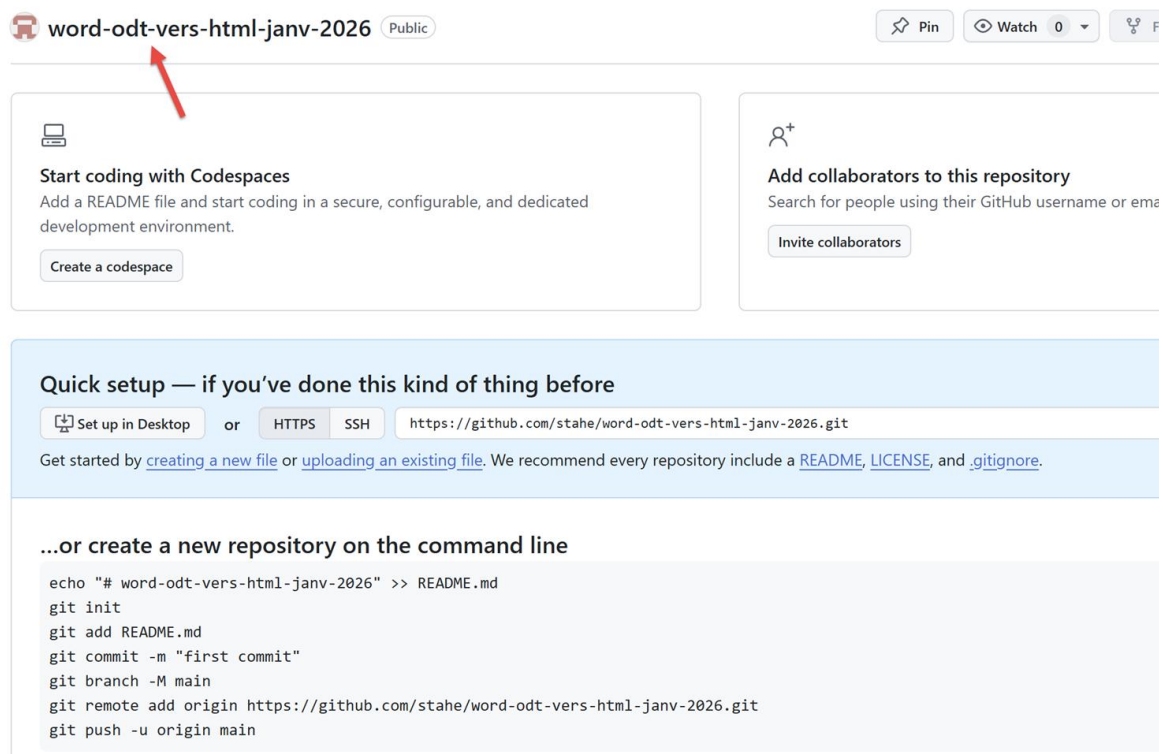
- In [1], use the name you entered in [config.py]:

```
18. "repo_url": "https://github.com/stahe/word-odt-vers-html-janv-2026",
```

- In [2], you can also enter the same information as in [config.py]:

```
19. "site_description": "Convert a Word or ODT document into a static HTML site using Gemini 3  
and ChatGPT 5.2 AI,"
```

- In [3], confirm the creation of your GitHub repository;

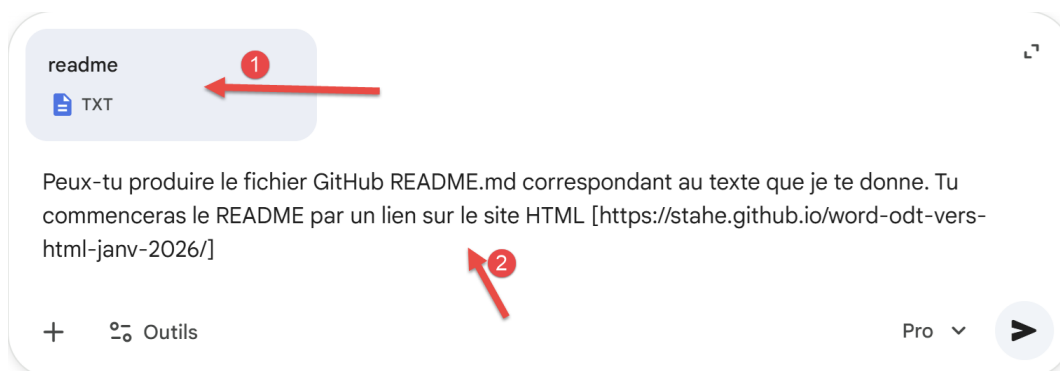


It is recommended that you create a [README.md] file for each GitHub repository, which will be displayed in place of the screen above. It must be written in Markdown, which isn't exactly straightforward. So we'll write a text file containing what we'd like to see in the README. Then we'll ask the AI to generate the corresponding README.md file.

The text file [readme.txt] will be as follows:

1. In this article, we aim to provide readers with a Python converter for converting Word or ODT documents into a static HTML site.
- 2.
3. This converter was built by the Gemini 3 AI.
- 4.
5. When the ODT document is suitable, the converter generates an HTML site via MkDocs that has the professional look of sites produced by Pandoc.
- 6.
7. I used Gemini 3 with a Pro subscription.

The prompt for Gemini is as follows:



- In [1], the attached file [readme.txt];
- In [2], the prompt;

Copy the response Gemini gives you into the [README.md] file in your working folder. Here is the response Gemini gave me:

```
1. # Word/ODT to HTML Site Converter (MkDocs)
2.
3.  **[View the generated demo site](https://stahe.github.io/word-odt-vers-html-janv-2026/)**
4.
5. ---
6.
7. ##  Description
8.
9. This project aims to provide readers with a Python converter for converting Word or ODT
  documents into a static HTML website.
10.
11. When the ODT document is suitable, the converter generates an HTML site using **MkDocs** that
  has the professional look of sites produced by Pandoc.
12.
13. ##  Creation Context
14.
15. This converter was built entirely by the **Gemini 3** AI (with a pro subscription). It is the
  result of successive iterations to finely manage the structure of ODT (OpenDocument Text)
  documents.
16.
17. ##  Features
18.
19. The `convert.py` script performs the following actions:
20.
21. * **ODT to Markdown Conversion**: Analyzes the `.odt` (XML) file to extract its structure.
22. * **Heading Management**: Automatically generates the Table of Contents (TOC) and sidebar
  navigation.
23. * **Code Blocks**: Automatic language detection, syntax highlighting, and **precise line
  numbering** (`start-value` attributes).
24. * **Lists**: Support for bulleted and numbered lists with correct indentation.
25. * **Formatting**: Support for *bold*, *italic*, *underline*, and *highlighting* (preserving
  original colors).
26. * **Images**: Automatic extraction and embedding of images contained in the document.
27. * **Configuration**: Customization via a `config.py` file (footer, Google Analytics, etc.).
28.
29. ##  Installation
30.
31. ### Prerequisites
32.
33. * Python 3.x
34. * The following libraries:
35.
36. ```bash
37. pip install odfpy unicode mkdocs mkdocs-material
38.
39. ```
40.
41. ### Project Structure
42.
43. Make sure you have the following files:
44.
45. * `convert.py`: The conversion script.
46. * `config.py`: Your configuration file.
47. * `your-document.odt`: The source document.
48.
49. ##  Usage
50.
51. 1. **Conversion**
52. Run the script by specifying the source ODT file and the configuration file:
53. ```bash
54. python convert.py your-document.odt config.py
55.
56. ```
57.
58.
59. *This will create a `docs/` folder containing the Markdown files and an `mkdocs.yml` file.*
60. 2. **Preview**
61. To view the site locally:
62. ```bash
63. mkdocs serve
```

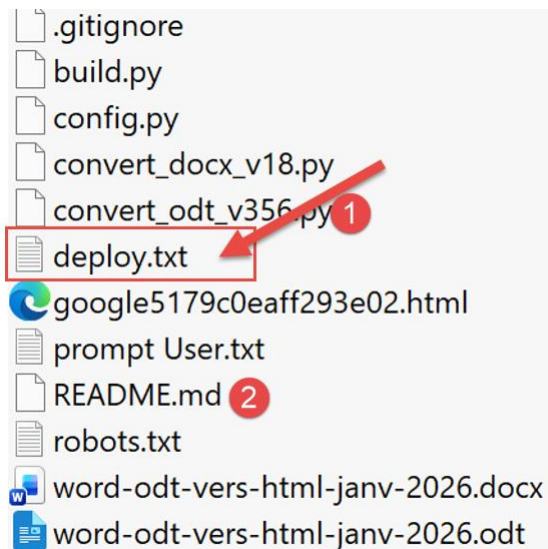
```

64.
65. ```
66.
67.
68. 3. Generation
69. To build the static site (folder `site/`):
70. ```bash
71. mkdocs build
72. ```
73.
74.
75.
76.
77. ## ⚙️ Configuration (`config.py`)
78.
79. The `config.py` file lets you control the site's appearance:
80.
81. * mkdocs: General site settings (title, description, Material theme).
82. * footer: Complete HTML code to customize the footer.
83. * code: Language detection rules for syntax highlighting.
84. * extra: Google Analytics (GA4) configuration.
85.
86. ## 📄 License
87.
88. This tutorial course written by Serge Tahé is made available to the public under the terms
  of the:
89. *Creative Commons Attribution - Non-Commercial - ShareAlike 3.0 Unported License.*

```

He provided me with a very comprehensive README. That's because Gemini is very familiar with this project, which we've been working on for weeks. You'll likely have a less detailed [README.md].

Now let's return to our working directory:



- In [2], the README file you just modified;
- In [1], the [deploy.txt] file explains how to export your HTML site to your GitHub repository;

The contents of the [deploy.txt] file are as follows:

```

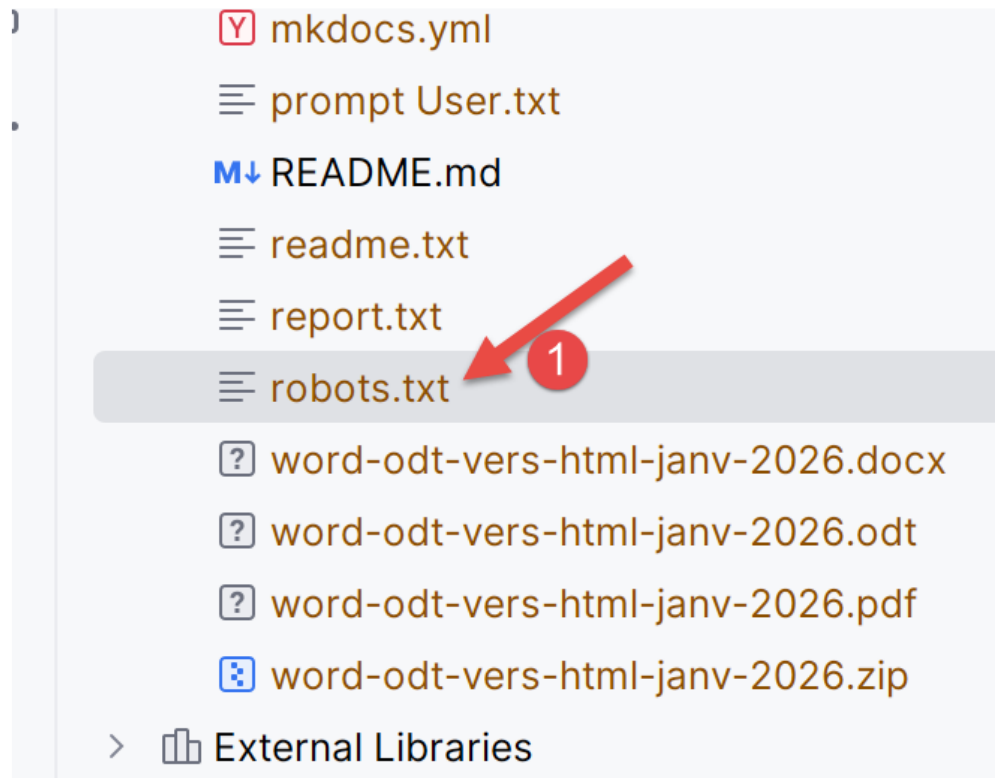
1. git init
2. git branch -M main
3. # Check .gitignore first!
4. git add .
5. # first commit
6. git commit -m "Initial commit: MkDocs source"
7. git remote add origin https://github.com/stahe/word-odt-vers-html-janv-2026.git
8. git push -u origin main
9. python -m mkdocs gh-deploy

```

This sequence of commands will export your HTML site to your GitHub repository. You will need to modify line 7 with the URL of your own GitHub repository, found in the [config.py] file:

```
20. "repo_url": "https://github.com/stahe/word-odt-vers-html-janv-2026",
```

This URL is also present in the [robots.txt] file:



The contents of the [robots.txt] file are as follows:

```
21. 1. User-agent: *
22. 2. Allow: /
23. 3. Sitemap: https://stahe.github.io/word-odt-vers-html-janv-2026/sitemap.xml
```

On line 3, enter your site's URL, the same one used in the [config.py] file. This file is not used during local site building, but it will be used once the site is hosted on GitHub.

The command sequence in [deploy.txt] uses a program called Git. You must install it [Git - Install for Windows].

Once that's done, check the file named [.gitignore] in your working directory. It tells Git which files to ignore. My [.gitignore] file is as follows:

```
1. # 1. We ignore ALL files and folders in the project
2. *
3.
4. # 3. We make an exception (!) for the single file you want to keep
5. !README.md
```

It's extremely simple. We ignore all files (line 2) except the [README.md] file (line 5). GitHub is designed to host development projects. Typically, the entire development project is uploaded to GitHub. We, however, simply want to upload an HTML site, not a development project. The only file we want to export to our GitHub repository is the [README.md] file, which explains to visitors what our HTML site contains.

Now, in your terminal, type the following sequence of commands in the order specified by [deploy.txt], up to command 8. Do not run command 9 for now.

```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git init
2. Initialized empty Git repository in C:/Data/st-2025\GitHub Pages\word-odt-vers-html\v2/.git/
3. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> git branch -M main
4. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> git add .
5. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git commit -m "Initial commit: Source MkDocs"
6. [main (root-commit) 7cba5b1] Initial commit: Source MkDocs
7. 1 file changed, 89 insertions(+)
8. create mode 100644 README.md
9. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> git remote add origin https://github.com/stahe/word-odt-vers-html-janv-2026.git
10. PS C:\Data\st-2025\GitHub Pages\word-odt-to-html\v2> git push -u origin main
11. Enumerating objects: 3, done.
12. Counting objects: 100% (3/3), done.
13. Delta compression using up to 8 threads
14. Compressing objects: 100% (2/2), done.
15. Writing objects: 100% (3/3), 1.70 KiB | 1.70 MiB/s, done.
16. Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
17. To https://github.com/stahe/word-odt-vers-html-janv-2026.git
18. * [new branch]      main -> main
19. The 'main' branch has been set up to track 'origin/main'.

```

Control-click the URL on line 17. This will take you to your GitHub repository:

The screenshot shows a web browser displaying the GitHub repository page for `stahe/word-odt-vers-html-janv-2026`. The address bar contains the URL `https://github.com/stahe/word-odt-vers-html-janv-2026`, with a red arrow labeled '1' pointing to it. The repository page shows the 'main' branch with 1 commit. A red arrow labeled '2' points to the 'README' file link in the file list. The README content is visible, titled 'Convertisseur Word/ODT vers Site HTML (MkDocs)', with a link to a demo site and a description of the project.

- In [1], the URL of your GitHub repository;
- In [2], the new README.md;

Now let's move on to line 9 of the [deploy.txt] file. This is the command that exports the HTML site to GitHub:

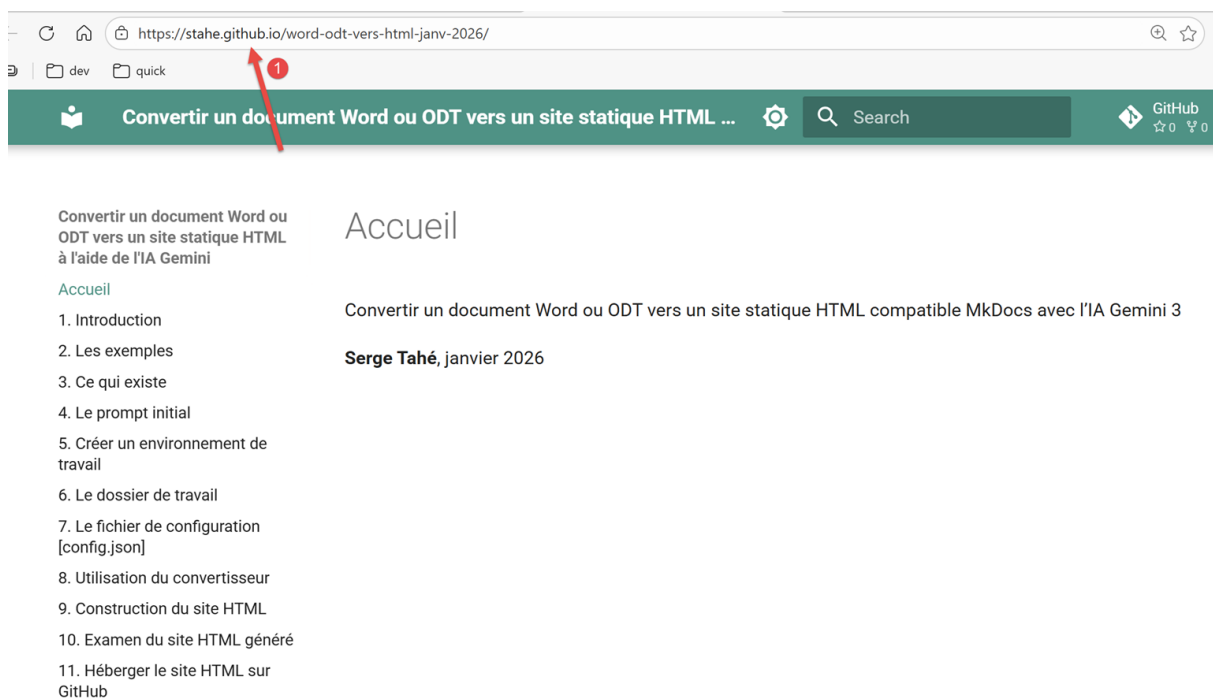
`https://tahe.developpez.com`


```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python -m mkdocs gh-deploy
2. INFO      - Cleaning site directory
3. INFO      - Building documentation to directory: C:\Data\st-2025\GitHub Pages\word-odt-vers-
html\v2\site
4. INFO      - Doc file 'les-exemples.md' contains a link '#_Les_exemples', but there is no such
anchor on this page.
5. INFO      - Documentation built in 1.79 seconds
6. WARNING   - Version check skipped: No version specified in previous deployment.
7. INFO      - Copying 'C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\site' to the 'gh-pages'
branch and pushing to GitHub.
8. Enumerating objects: 91, done.
9. Counting objects: 100% (91/91), done.
10. Delta compression using up to 8 threads
11. Compressing objects: 100% (85/85), done.
12. Writing objects: 100% (91/91), 1.64 MiB | 2.01 MiB/s, done.
13. Total 91 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
14. remote: Resolving deltas: 100% (9/9), done.
15. remote:
16. remote: Create a pull request for 'gh-pages' on GitHub by visiting:
17. remote:      https://github.com/stahe/word-odt-vers-html-janv-2026/pull/new/gh-pages
18. remote:
19. Tohttps://github.com/stahe/word-odt-vers-html-janv-2026.git
20. * [new branch]      gh-pages -> gh-pages
21. INFO      - Your documentation should shortly be available at: https://stahe.github.io/word-
odt-vers-html-janv-2026/

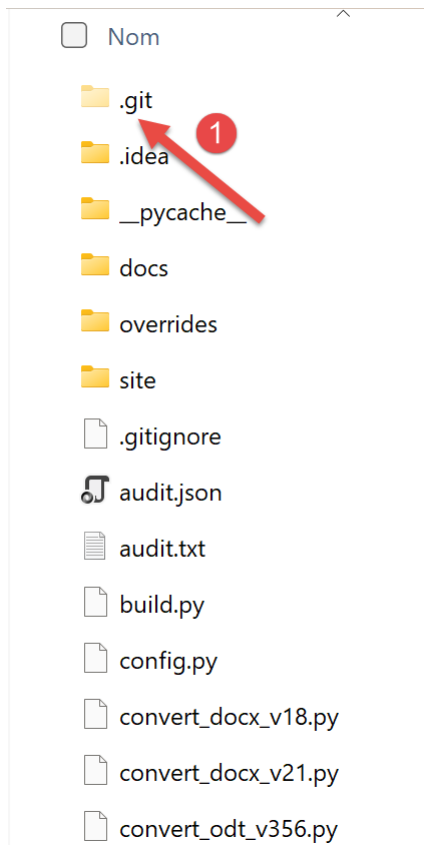
```

Ctrl-click on the URL in line 21. This will take you to your new HTML site on GitHub:



- In [1], you can see that you are displaying a website on GitHub;

It is extremely easy to make a mistake when running the commands in [deploy.txt]. It is then difficult to undo the changes because Git keeps a record of what you've done (wrong). To start over, view the working directory:



- In [1], delete the [.git] folder;

Then go back to PyCharm and run the series of commands from [deploy.txt] again.

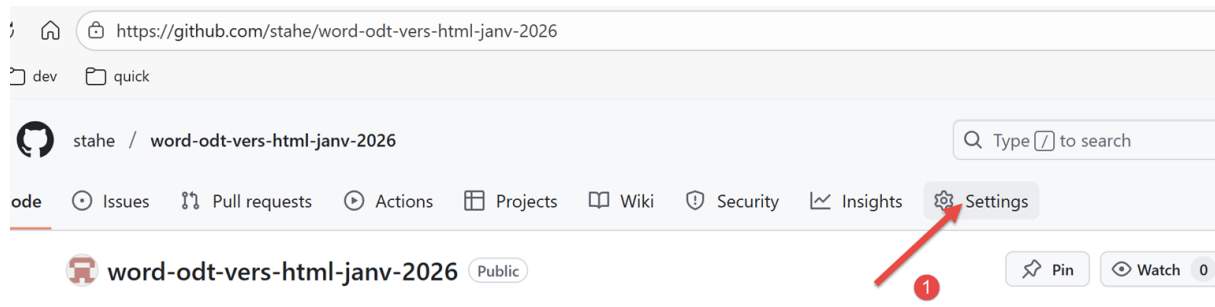
What should you do if you modify your ODT/DOCX document? Do the following 3 things:

1. Convert your ODT/DOCX document again using [convert];
2. build the HTML site with [build];
3. export the HTML site to GitHub using the command [python -m mkdocs gh-deploy]. This command is sufficient as long as you do not modify the [README.md] file. If you modify the [README.md] file, you will need to run a few more commands:

```
1. git add README.md
2. # commit
3. git commit -m "README update"
4. git push -u origin main
5. python -m mkdocs gh-deploy
```

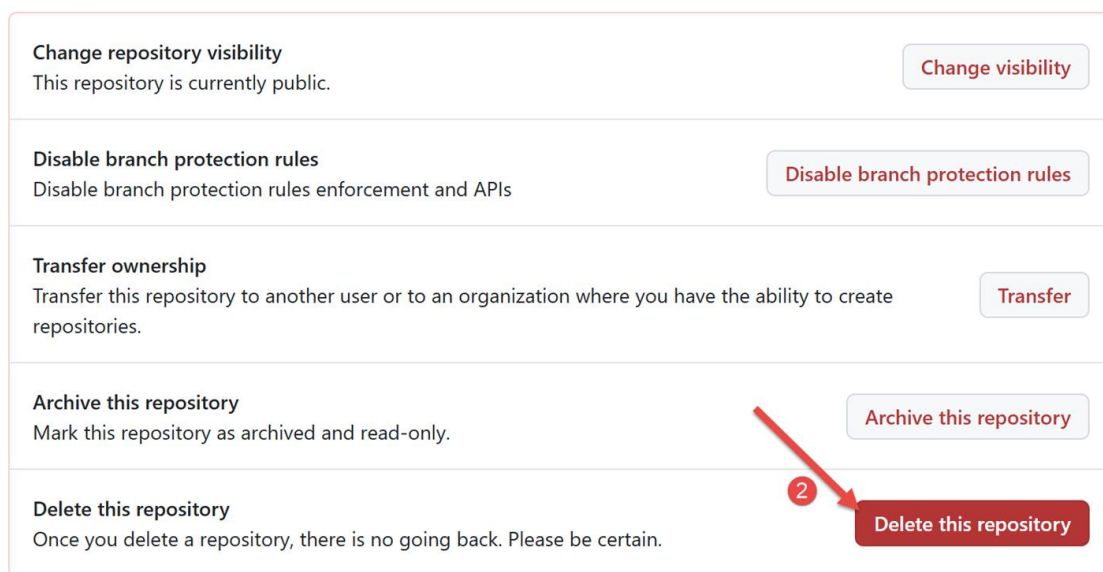
If you've only modified the README, only commands 1, 3, and 4 are necessary. Command 5 is unnecessary if you've already deployed the HTML site and it hasn't changed since then.

What if you want to start from scratch because things have gone haywire? You can delete your GitHub repository and repeat all the steps from Chapter 12. The option to delete a GitHub repository is well hidden:



- In [1], go to the repository settings;

Danger Zone



Scroll all the way to the bottom of the [settings] page. There you'll find the button to delete your repository [2].

13. Track your site with Google Analytics and Google Search Console

If you host your site on GitHub, you'll likely want to know how many visitors your site gets. For that, you'll need a Google Analytics (GA) tag. We'll ask ChatGPT how to create a GA tag:

comment obtient-t-on un marqueur Google Analytics pour un site web ?

Pour un site web, le **marqueur Google Analytics** correspond aujourd'hui à l'**identifiant de mesure GA4** (format **G-xxxxxxxxx**). Voici la procédure complète et à jour pour l'obtenir.

1. Créer ou utiliser un compte Google Analytics

1. Accédez à Google Analytics et connectez-vous avec votre compte Google.
2. Si aucun compte n'existe encore :
 - Cliquez sur **Créer un compte**
 - Donnez un nom au compte (ex. : *Entreprise X*)
3. Si un compte existe déjà, sélectionnez-le.

2. Créer une propriété GA4

1. Dans **Administration** (icône engrenage, en bas à gauche),
2. Colonne **Propriété** → **Créer une propriété**,
3. Renseignez :
 - Nom de la propriété
 - Fuseau horaire
 - Devise
4. Validez.

Depuis plusieurs années, **GA4 est le seul type de propriété disponible**. Les anciennes propriétés Universal Analytics ne sont plus utilisables.

3. Créer un flux de données Web

1. Toujours dans **Administration**,
2. Colonne **Propriété** → **Flux de données**,
3. Cliquez sur **Ajouter un flux** → **Web**,
4. Saisissez :
 - L'URL du site (ex. `https://www.monsite.fr`)
 - Un nom de flux
5. Validez.



- In [1], enter the URL of your website on GitHub;

4. Récupérer le marqueur (ID de mesure)

Après la création du flux web :

- Un **ID de mesure** est affiché, au format :

CSS

 Copier le code

G-XXXXXXXXXX

2

👉 C'est ce marqueur Google Analytics

Il identifie de manière unique votre site dans GA4.

- In [2], note your site's tag;

Add this tag to the configuration file [config.py]:

```
1. "extra": {
2.     "analytics": {
3.         "provider": "google",
4.         "property": "G-XXXXXXXXXX"
5.     }
6. },
```

On line 4, enter your tracking code. MkDocs will automatically insert it into all your HTML pages when it generates the HTML site. Therefore, you do not need to follow the other instructions from ChatGPT that show how to install the GA tracking code in your HTML pages.

We'll ask ChatGPT for specific instructions to verify that the tag is working:

 donne moi des instructions plus précises pour vérifier que le marqueur fonctionne. Donne moi l'URL à utiliser.

Voici une procédure **opérationnelle et vérifiable pas à pas** pour contrôler que votre marqueur GA4 fonctionne, avec l'**URL exacte à utiliser**.

Méthode recommandée : vérification via le rapport *Temps réel*

1. Accéder à Google Analytics

Ouvrez directement l'interface de Google Analytics à l'adresse suivante :

<https://analytics.google.com/> ➤

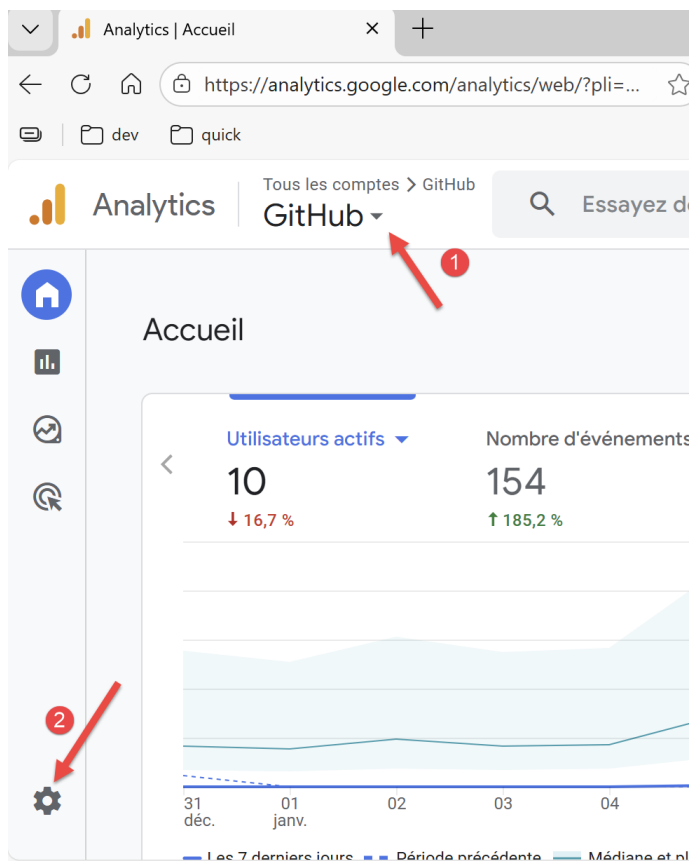
Connectez-vous avec le compte Google qui possède la propriété GA4.

2. Sélectionner la bonne propriété

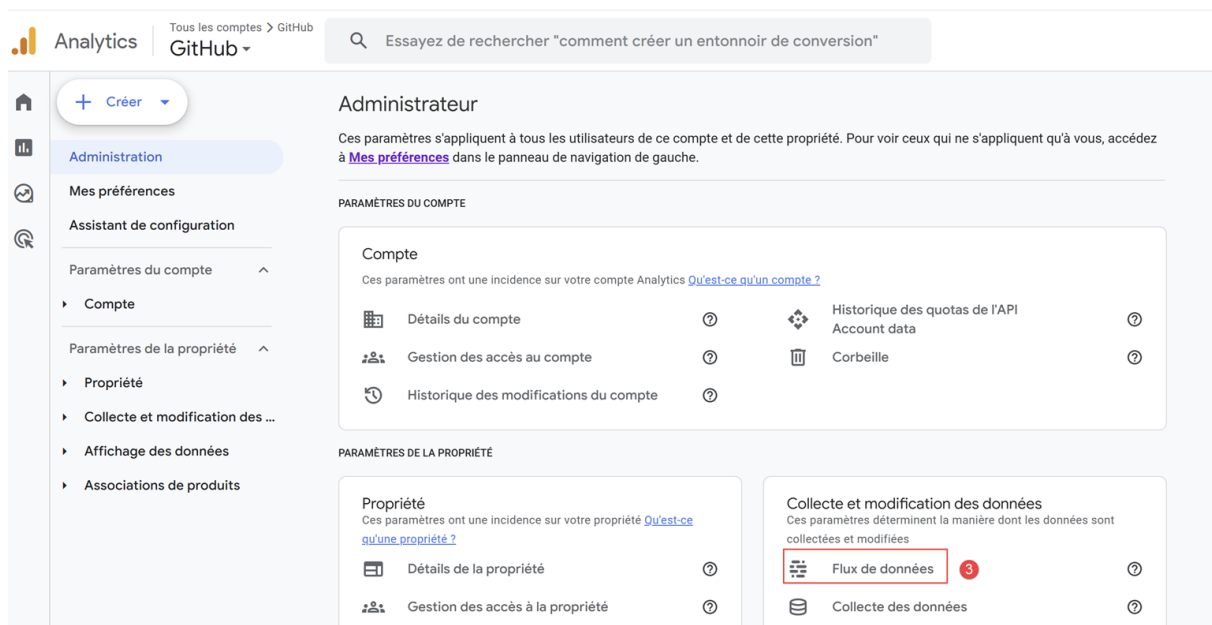
En haut à gauche de l'interface :

1. Vérifiez que la **propriété GA4** correspondant à votre site est bien sélectionnée.
2. Si nécessaire, changez-la via le sélecteur de compte/propriété.

The previous step isn't clear enough. Here are some screenshots to help you:



- In [1], select the account you want to track (in case you have multiple accounts);
- In [2], select the admin button;



- In [3], select [Data Stream];

Flux de données

[Toutes les vues](#)
[iOS](#)
[Android](#)
[Web](#)

[Ajouter un flux](#)

GitHub https://stahe.github.io		13112486249	La propriété a reçu du trafic au cours des dernières 48 heures.	>
--	--	-------------	---	---

- In [4], click on the data feed;

× Détails du flux Web

La collecte de données a été active au cours des 48 dernières heures.

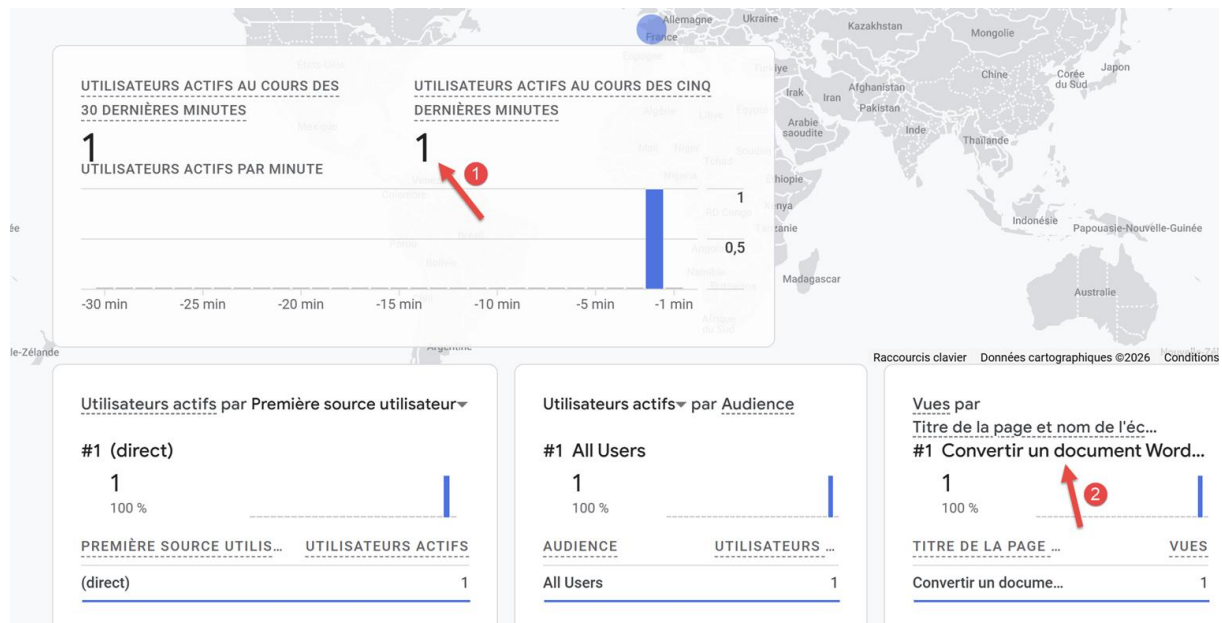
Détails du flux

NOM DU FLUX	URL DE FLUX	ID DE FLUX	ID DE MESURE
GitHub	https://stahe.github.io	13112486249	G-EBZC

- In [5], you will find the GA4 tag you created;
- In [6], the stream ID. This is different from the measurement ID;

Let's go back to the GA homepage:

- In [1], select the real-time view of visits;



The Google Analytics GA4 tag is working properly.

Now we'll introduce another tool for monitoring your site [Google Search Console]. Let's ask ChatGPT what this tool is used for:

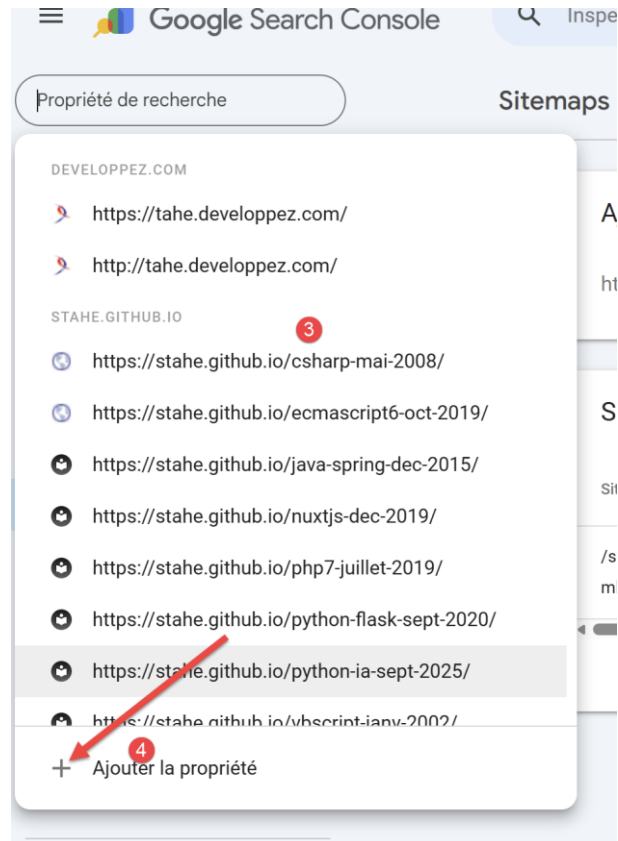
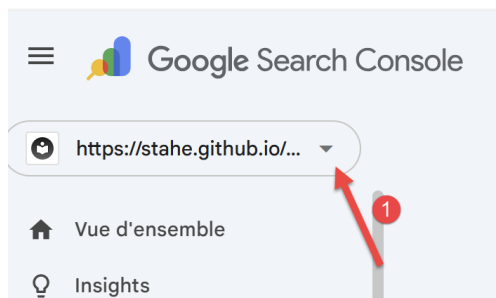
Conclusion

Google Search Console sert à :

- comprendre **comment Google perçoit votre site**,
- améliorer votre **référencement naturel (SEO)**,
- détecter rapidement les **problèmes bloquants**,
- piloter votre visibilité dans Google Search.

I almost never use this tool. I only use it to encourage the Google search engine to crawl and index the site so that users can find it.

The tool's URL [<https://search.google.com/search-console>]:



- In [1], the list of sites that have been submitted to Google;
- In [3], the list of your sites;
- In [4], add your new site;

Sélectionnez le type de propriété



Domaine

- Toutes les URL de tous les sous-domaines (m. www. ...)
- Toutes les URL sur https et http
- Validation DNS obligatoire

Saisissez un domaine ou sous-domaine

CONTINUER

ou



Préfixe de l'URL

- Seules les URL de l'adresse saisie
- Seules les URL du protocole spécifié
- Différentes méthodes de validation acceptées

Saisissez une URL

CONTINUER

- In [5], enter your site's URL. It's in the [config.py] file:

```
24.     "site_url": "https://stahe.github.io/word-odt-vers-html-janv-2026/",
```

- In [6], proceed to the next step;

Normally, at this step, Google Search Console will offer you a [googlexxxxx.html] file that you need to download. I didn't see this screen because I already had this file on the deployed site. It is one of the two files declared in [config.py]

```
1.     "files_to_copy": [
2.         "google5179xxxxx.html",
3.         "robots.txt"
4.     ]
```

Place the [googlexxxxx.html] file in the root of your working directory and enter its name on line 2 above in the [config.py] file. The [convert] script handles placing the two files above in the root of the MkDocs site it generates. The [build] script will place them in the root of the generated HTML site.

Once you have copied the [googlexxxxx.html] file to the root of your working directory, check the contents of the [robots.txt] file in your working directory:

Once you have copied the [googlexxxxx.html] file to the root of your working directory, check the contents of the [robots.txt] file:

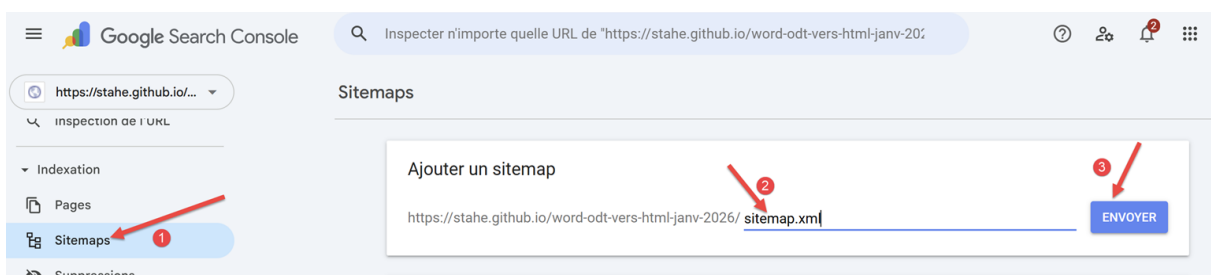
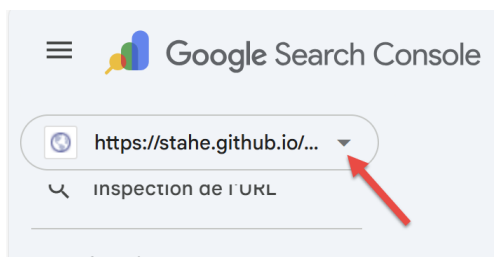
```
25.     1. User-agent: *
26.         2. Allow: /
27.         3. Sitemap: https://stahe.github.io/word-odt-vers-html-janv-2026/sitemap.xml
```

On line 3, make sure the URL is your site's URL, the same as in the [config.py] file.

Deploy your website to GitHub by entering the following three commands in your Python terminal:

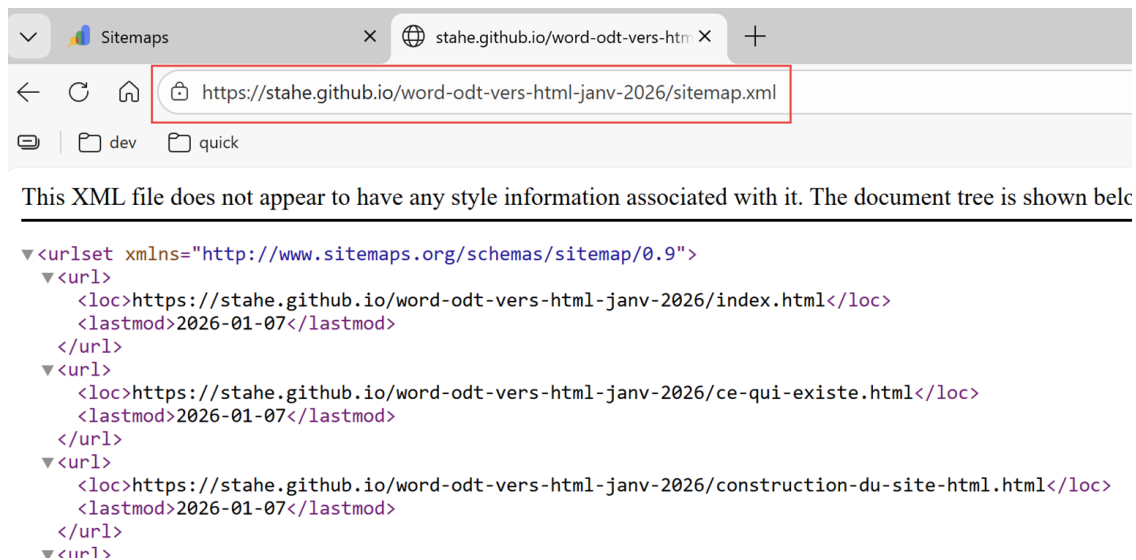
```
1. python .\convert.py .\word-odt-to-HTML.odt config.py
2. python .\build.py
3. python -m mkdocs gh-deploy
```

Once this is done, go back to [Google Search Console] and select the property you created earlier:



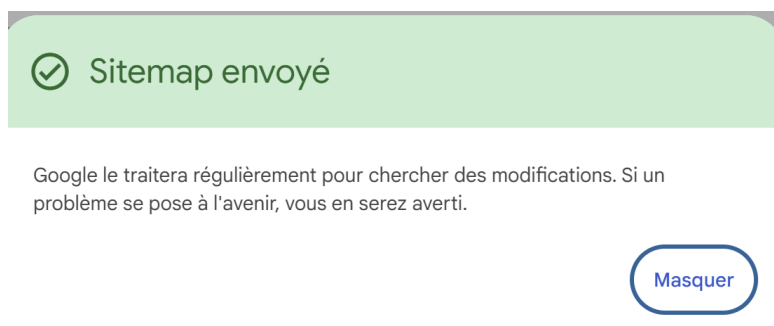
- In [1], select the [Sitemaps] option;
- In [2], type [sitemap.xml];
- In [3], validate the URL;

The [sitemap.xml] file is a file that Mkdocs placed in the root directory of the HTML site exported to GitHub. You can verify its presence by typing its URL directly:

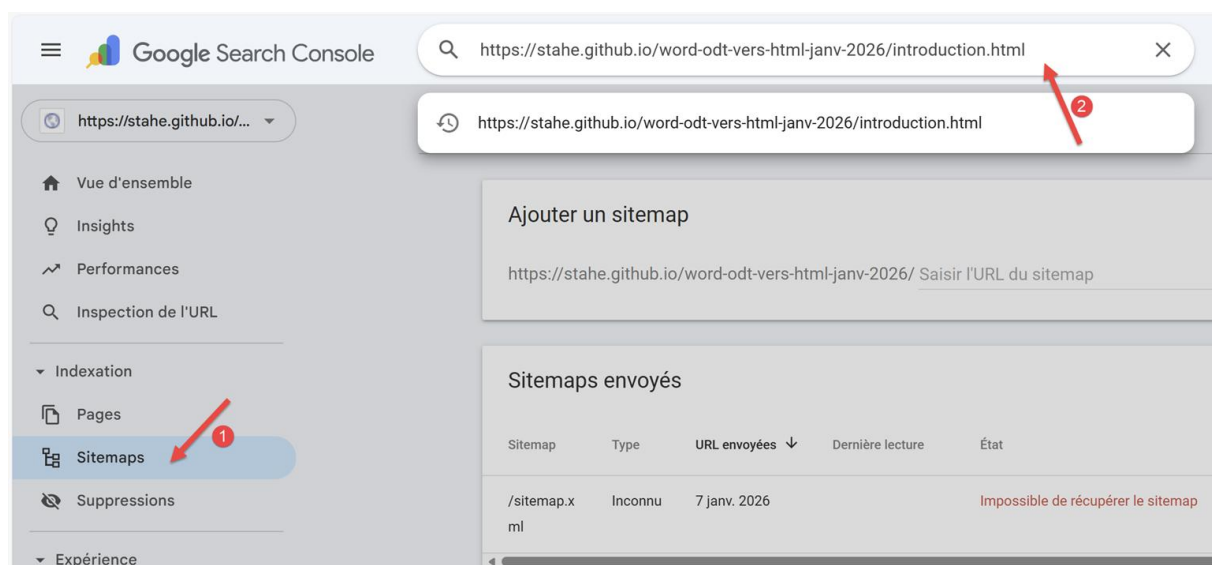


The [sitemap.xml] file lists all the HTML pages on the site.

If everything goes well, you'll see the following screen:

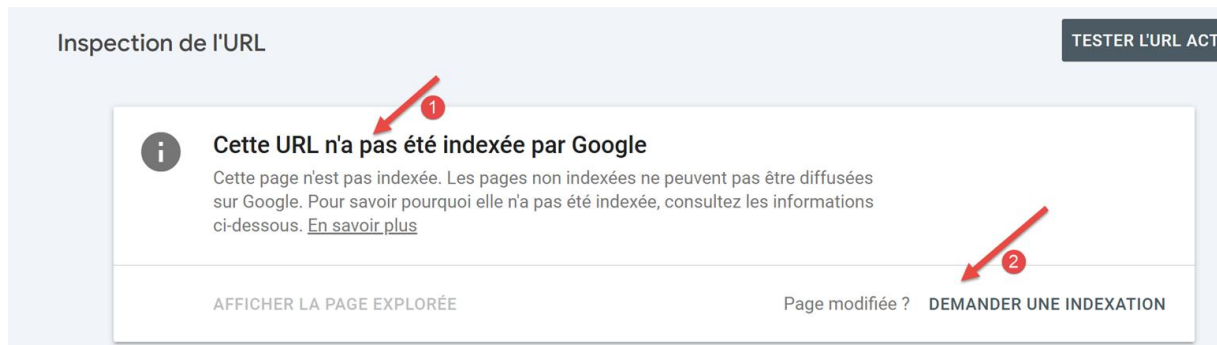


That's it. You'll need to index the site's pages yourself:



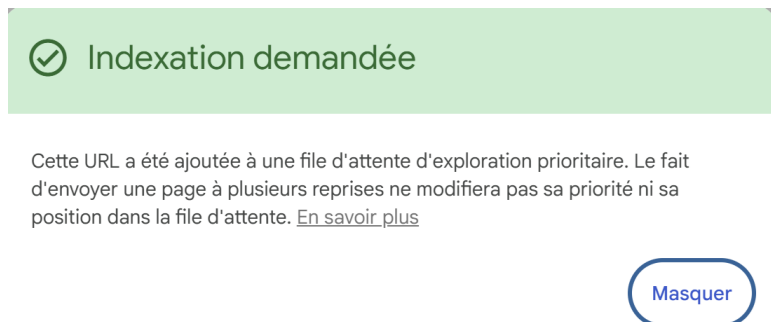
- In [2], enter the URL of one of your site's pages;

Here, the result is as follows:



- In [1], Google says the page isn't indexed yet;
- In [2], you can request that it be indexed;

Google Search Console then checks whether the requested page can be indexed. If so, you'll see the following message:



You can do this for all the pages on your site to ensure that it is properly indexed by Google's search engine.

14. Conclusion

In this document, we have demonstrated a converter that transforms ODT documents created by LibreOffice or DOCX documents from Word into a high-quality static HTML site.

In the chapter [Examples in this document], we presented the structures correctly handled by the Gemini/ChatGPT converter.

We have also shown how to export the HTML sites created by the ODT and DOCX converters to GitHub (see the chapter "Hosting the HTML Site on GitHub")

Outside the structures outlined in the examples section, nothing is guaranteed. It is likely that the converter will produce errors. You will then need to start a new chat with Gemini or ChatGPT to resolve these errors. The simplest approach is to provide the AI with the current converter and its configuration file. Then ask the AI if it understands the code you've provided. This forces it to analyze the code. It will always respond that it fully understands the code. That's when you should ask it for the desired changes. For both Word and LibreOffice, you can assist the AI with macros. The AI cannot "see" the ODT or DOCX document being processed. It therefore lacks certain information. As a result, the DOCX-to-HTML converter couldn't properly handle the numbering of rich code blocks. It couldn't read the line number of the first line in a code block to render it in HTML. It always started the code at line 1, even if the code in the DOCX file started at line 12. So I used a macro. I placed the document cursor on the numbered line and asked ChatGPT to generate a VBA macro that would provide it with the characteristics of that line, enabling it to correctly generate the HTML numbering. ChatGPT did so, and I gave it the result of its macro. That solved the

problem: the retrieved information showed it that the line number was propagated by its style, not by the numbering icon. It then changed its algorithm, and it worked. So we can help AI move forward.

Another, simpler approach is to modify your document. When the converter produces an error, try modifying your document so that it uses only the chapter structures from the examples.